

JAVA BACKEND ARCHITECTURE

Interview Series

193 Focused Sub-Chapters across 19 Master Topics
Complete Interview Preparation for Senior Java/Architecture Roles

193
Sub-Chapters

19 Master
Topics

6 Phases

~20 Pages
Each

40+ Q&A;
Each

This series organises Java backend interview preparation into 193 focused sub-chapters across 19 master topics and 6 development phases. Chapter 1 (Core Java & JVM Internals) contains 5 dedicated sub-chapters on the Collections Framework — covering List, Set, Map, Queue/Deque, and framework internals/algorithms — reflecting the depth required at senior level. Every sub-chapter is a self-contained ~20-page PDF with documentation, runnable Java code, 30–40 interview Q&A;, and 4 exercise types (coding challenges, debug-and-fix, design problems, MCQ).

Phase	Master Chapters	Sub-Chapters	Chapter Nos.
Phase 1: Foundations	2	23	1–2
Phase 2: Application Framework & Data Layer	5	50	3–7
Phase 3: Application Security	1	10	8–8
Phase 4: Application Capabilities	5	50	9–13
Phase 5: Performance & Architecture	2	20	14–15
Phase 6: Infrastructure & Deployment	4	40	16–19

Phase 1: Foundations

Chapter 1 Core Java & JVM Internals (13 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
1.1	Java Primitives, Wrappers & Strings	Primitive types & sizes, autoboxing/unboxing, Integer cache (-128..127) pitfall, String immutability, String pool & intern(), StringBuilder vs StringBuffer, equals()/hashCode() contract (reflexive/symmetric/transitive)
1.2	Collections – Framework Overview & List	Full hierarchy: Iterable > Collection > List/Set/Queue/Deque; Map (separate). ArrayList (Object[] backed, O(1) get, 1.5x resize, amortized O(1) add), LinkedList (doubly-linked + Deque, O(1) ends, O(n) index), CopyOnWriteArrayList (snapshot copy on write, thread-safe reads, no CME). All key methods: add/get/set/remove/contains/indexOf/subList/sort/removeIf/List.of()
1.3	Collections – Set Implementations	HashSet (hash table, O(1), no order), LinkedHashSet (insertion-order, O(1)), TreeSet (red-black tree, sorted, O(log n), NavigableSet: floor/ceiling/headSet/tailSet/pollFirst/pollLast), EnumSet (bit-vector, fastest for enums). Set algebra: union (addAll), intersection (retainAll), difference (removeAll). All Set/SortedSet/NavigableSet methods with code examples
1.4	Collections – Map Implementations	HashMap (bucket array + linked list/tree@bin≥8, hash spreading h^h>>>16, load=0.75, resize doubles), LinkedHashMap (insertion/access order, LRU cache via removeEldestEntry+accessOrder=true), TreeMap (red-black tree, NavigableMap: floorKey/ceilingKey/headMap/tailMap/pollFirstEntry), ConcurrentHashMap (CAS + per-bin sync, no full-table lock, null key/value NOT allowed), EnumMap (array-backed O(1)), WeakHashMap (GC-able keys). All Map methods: put/get/getOrDefault/putIfAbsent/computeIfAbsent/merge/forEach/entrySet
1.5	Collections – Queue, Deque & BlockingQueue	PriorityQueue (min-heap, O(log n) offer/poll, O(1) peek, custom Comparator for max-heap), ArrayDeque (preferred Stack AND Queue – no sync overhead vs Stack, no node overhead vs LinkedList), all Deque methods: offerFirst/Last/pollFirst/Last/peekFirst/Last/push/pop. BlockingQueue: LinkedBlockingQueue/ArrayBlockingQueue/PriorityBlockingQueue — put() blocks if full, take() blocks if empty, producer-consumer pattern
1.6	Collections – Internals, Algorithms & Best Practices	Fail-fast iterators (modCount check → ConcurrentModificationException) vs fail-safe (CopyOnWriteArrayList/ConcurrentHashMap snapshot). Collections utility class: sort/binarySearch/reverse/shuffle/unmodifiableList/synchronizedList/frequency/disjoint/min/max/nCopies/singletonList. Choosing the right collection – decision table (15 use cases). Performance comparison table: time/space complexity for all operations across all implementations
1.7	Generics & Type System	Type erasure (compile-time only, erased to bounds at runtime), bridge methods, wildcards (? extends T upper bound, ? super T lower bound), PECS rule (Producer Extends Consumer Super), generic methods, bounded type params, cannot new T() or T[]
1.8	Exception Handling	Throwable hierarchy: Error (don't catch) vs Exception vs RuntimeException; checked vs unchecked; try-with-resources (AutoCloseable, reverse close order, suppressed exceptions); multi-catch; custom exceptions; finally behaviour; anti-patterns (swallowing, flow-control via exception)
1.9	Java 8 – Lambdas, Streams & Optional	Functional interfaces (Function/Predicate/Consumer/Supplier/BinaryOperator), method references (Class::method, instance::method, Class::new), Stream lazy evaluation (intermediate vs terminal ops), key Collectors (groupingBy/joining/toMap/counting/partitioningBy), Optional (orElse vs orElseGet lazy, map/flatMap/filter/orElseThrow)
1.10	Java 9–21 Modern Features	Records (Java 16 – immutable data carrier, auto constructor/getters/equals/hashCode/toString, compact constructor), sealed classes (Java 17 – restrict subtypes, exhaustive switch), pattern matching instanceof (Java 16), switch expressions (Java 14), text blocks (Java 15), var (Java 10), virtual threads / Project Loom (Java 21 – JVM-managed ~1KB, millions concurrent, IO-bound)

No.	Sub-Chapter Title	Key Topics Covered
1.11	Multithreading Fundamentals	Thread lifecycle (NEW/RUNNABLE/BLOCKED/WAITING/TIMED_WAITING/TERMINATED), synchronized (method vs block, intrinsic monitor), volatile (visibility not atomicity, happens-before), wait()/notify()/notifyAll() (must hold monitor), Java Memory Model happens-before rules, deadlock (conditions: mutual exclusion/hold-and-wait/no-preemption/circular-wait), prevention strategies
1.12	java.util.concurrent Utilities	ReentrantLock (tryLock/lockInterruptibly/fair), ReadWriteLock/StampedLock (optimistic read), AtomicInteger/CAS/ABA problem (AtomicStampedReference fix), CompletableFuture (thenApply/thenCompose/thenCombine/exceptionally/allOf/anyOf), CountdownLatch/CyclicBarrier/Semaphore, ThreadLocal (isolation + memory leak in thread pools – always remove()), ExecutorService sizing (CPU-bound N+1, IO-bound $N \times (1 + \text{wait}/\text{compute})$)
1.13	JVM Internals – Class Loading, JIT & GC	Class loader delegation (Bootstrap>Platform>Application, parent-first), loading phases (Load→Link→Init), JIT tiered compilation (C1 fast+profiling → C2 aggressive: inlining/escape analysis/loop unrolling), G1GC (regions, concurrent marking, mixed GC, MaxGCPauseMillis), ZGC (sub-ms, concurrent compaction, colored pointers), Metaspace, heap dump analysis (jmap + Eclipse MAT)

Chapter 2 Design Patterns & Design Principles (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
2.1	SOLID Principles	SRP (one reason to change), OCP (open for extension, closed for modification), LSP (subtypes must honour supertype contracts), ISP (no fat interfaces), DIP (depend on abstractions). Each with Java violations and correct examples, real-world Spring usage
2.2	Creational Patterns	Singleton (double-checked locking, enum singleton, thread safety), Factory Method (defer instantiation to subclass), Abstract Factory (family of products), Builder (fluent API, Lombok @Builder, immutable construction), Prototype (clone(), deep vs shallow), Object Pool (connection pool)
2.3	Structural Patterns – Part 1	Adapter (class vs object adapter, legacy API integration), Bridge (abstraction + implementation hierarchies, JDBC driver model), Composite (tree structures, File/Directory, UI components), Decorator (InputStream chain, Spring transaction proxy)
2.4	Structural Patterns – Part 2	Facade (subsystem simplification, SLF4J over logging frameworks), Flyweight (String pool, Integer cache, shared immutable state), Proxy (JDK dynamic proxy via InvocationHandler, CGLIB subclass proxy, Spring AOP proxy, remote proxy)
2.5	Behavioral Patterns – Part 1	Chain of Responsibility (servlet filter chain, Spring Security filter chain, ordered handlers), Command (encapsulate action, undo/redo, job queue, macro recording), Iterator (Iterable/Iterator contract, fail-fast, custom iterators), Mediator (event bus, Spring ApplicationEventPublisher)
2.6	Behavioral Patterns – Part 2	Observer (Spring @EventListener, @Async events, loose coupling), State (order state machine: PENDING→CONFIRMED→SHIPPED→DELIVERED, state-specific behaviour), Strategy (Comparator, pluggable payment/discount/sort algorithms, @ConditionalOn), Template Method (JdbcTemplate, AbstractList, hook methods)
2.7	Behavioral Patterns – Part 3	Memento (undo history, snapshot without exposing state), Visitor (double dispatch, AST processing, Jackson JsonNode visitor, report generation), Interpreter (DSL parsing, SpEL), Null Object (avoid null checks, Optional comparison)
2.8	Enterprise & Architectural Patterns	Repository (abstract data access, Spring Data), DTO (data transfer, MapStruct mapping, avoiding entity exposure), Service Layer (transaction boundary, orchestration), Unit of Work (JPA PersistenceContext), Data Mapper vs Active Record, Outbox Pattern

No.	Sub-Chapter Title	Key Topics Covered
2.9	Domain-Driven Design (DDD) Basics	Bounded Context (explicit model boundaries, context map), Aggregate & Aggregate Root (consistency boundary, transactional invariants), Value Object (immutable, equality by value), Domain Event, Domain Service, Application Service, Anti-Corruption Layer (ACL)
2.10	Clean Architecture & Refactoring	Hexagonal Architecture (Ports & Adapters, inbound/outbound ports), Clean Architecture layers (Entities→Use Cases→Interface Adapters→Frameworks), Dependency Rule (inner layers know nothing of outer), code smells (God Class, Feature Envy, Long Method, Primitive Obsession), refactoring: Extract Method, Move Method, Replace Temp with Query, Strangler Fig pattern

Phase 2: Application Framework & Data Layer

Chapter 3 Spring Framework Internals (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
3.1	IoC Container & Dependency Injection	BeanFactory vs ApplicationContext, XML vs annotation vs Java @Configuration, @Component/@Service/@Repository/@Controller stereotypes, @Autowired/@Inject/@Qualifier, constructor (preferred) vs field vs setter injection, @Primary, @Lazy
3.2	Bean Lifecycle & Scopes	Scopes: singleton/prototype/request/session/application/websocket. Lifecycle: constructor → @PostConstruct → use → @PreDestroy. BeanPostProcessor (before/after init), BeanFactoryPostProcessor, @DependsOn, SmartInitializingSingleton
3.3	Spring AOP & Proxy Mechanism	AOP concepts (Aspect/Pointcut/JoinPoint/Advice/Weaving), JDK dynamic proxy (interface-based) vs CGLIB (subclass-based), @Aspect+@Pointcut+@Around/@Before/@After, self-invocation bypass issue, execution() vs @annotation() pointcut expressions
3.4	Spring MVC & REST API	@Controller vs @RestController, @RequestMapping/@GetMapping/@PostMapping, @PathVariable/@RequestParam/@RequestBody/@ResponseBody, ResponseEntity, content negotiation, @ControllerAdvice+@ExceptionHandler, HandlerInterceptor, message converters (Jackson)
3.5	Spring Boot Auto-configuration & Actuator	@SpringBootApplication = @Configuration+@EnableAutoConfiguration+@ComponentScan, spring.factories/AutoConfiguration.imports, @ConditionalOnClass/@ConditionalOnMissingBean/@ConditionalOnProperty, Actuator (health/metrics/env/beans/loggers/heapdump), Micrometer metrics, custom health indicators
3.6	Spring Data & Repository Pattern	CrudRepository/JpaRepository/PagingAndSortingRepository, derived query methods (findByNameAndAge), @Query (JPQL + nativeQuery=true), @Modifying+@Transactional, Specification (Criteria API wrapper), interface/class/dynamic projections, QueryDSL, Auditing (@CreatedDate/@LastModifiedBy)
3.7	Spring Transaction Management	@Transactional internals (AOP proxy, TransactionSynchronizationManager), propagation levels (REQUIRED/REQUIRES_NEW/NESTED/SUPPORTS/NOT_SUPPORTED/MANDATORY/NEVER), isolation levels, readOnly optimisation (Hibernate flush mode MANUAL), rollback rules (checked exceptions don't rollback by default), self-invocation pitfall
3.8	Spring Security Architecture	SecurityFilterChain, DelegatingFilterProxy, FilterChainProxy, AuthenticationManager→ProviderManager→AuthenticationProvider chain, UserDetailsService, PasswordEncoder (BCrypt), SecurityContextHolder (ThreadLocal strategy), method security (@PreAuthorize SpEL, @Secured, @RolesAllowed), CSRF (SameSite + token)
3.9	Spring Events & Integration	ApplicationEvent + @EventListener, @Async events (SimpleAsyncTaskExecutor), ordered listeners (@Order), Spring Integration (channels/endpoints/transformers), Spring Batch (Job/Step/ItemReader/Processor/Writer, chunk-oriented, fault tolerance), @Scheduled (fixedRate/fixedDelay/cron), Spring Retry (@Retryable)
3.10	Spring Testing	@SpringBootTest (full context, webEnvironment), @WebMvcTest (MVC layer only, MockMvc), @DataJpaTest (JPA + H2), @MockBean/@SpyBean (Mockito in context), Testcontainers (real PostgreSQL/Redis/Kafka in tests), @Sql (setup scripts), RestAssured (API testing), WireMock (stub external HTTP)

Chapter 4 JPA & ORM Internals (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
4.1	JPA Entity Lifecycle & Persistence Context	Entity states: Transient (no ID, not tracked) → Managed (in PC, dirty-checked) → Detached (ID exists, not tracked) → Removed (queued for DELETE). PersistenceContext as first-level cache (per-Session). EntityManager: persist/merge/remove/find/getReference/flush/clear/detach. Extended vs transaction-scoped PC
4.2	Hibernate Architecture	SessionFactory (heavy, application-scoped, thread-safe) vs Session (lightweight, not thread-safe, short-lived), EntityManagerFactory wraps SessionFactory, StatelessSession for bulk ops (no PC/events/cache), Hibernate statistics (enable for monitoring), HibernateJpaDialect
4.3	Entity Mappings	@OneToOne/@OneToMany/@ManyToOne/@ManyToMany with @JoinColumn/@JoinTable, cascade types (ALL/PERSIST/MERGE/REMOVE/REFRESH/DETACH), orphanRemoval, @Embeddable/@Embedded (value objects), inheritance strategies: SINGLE_TABLE (discriminator) / JOINED (separate tables) / TABLE_PER_CLASS (no FK), @MappedSuperclass, @ElementCollection
4.4	JPQL, Native Queries & Criteria API	JPQL syntax (SELECT e FROM Employee e JOIN FETCH e.dept WHERE e.salary > :min), named queries (@NamedQuery, precompiled), native queries (@Query nativeQuery=true, SqlResultSetMapping), Criteria API (CriteriaBuilder/CriteriaQuery/Root/Join, type-safe), Metamodel (static EmployeeClass_)
4.5	Lazy Loading & N+1 Problem	FetchType.LAZY (default for @OneToMany/@ManyToMany) vs EAGER (default for @ManyToOne/@OneToOne), Hibernate proxy for lazy single associations, LazyInitializationException (detached entity), N+1: detect with Hibernate statistics, fix with JOIN FETCH / @EntityGraph / batch fetching (hibernate.default_batch_fetch_size) / DTO projection
4.6	JPA Caching – L1 & L2	L1 cache (PersistenceContext, per-Session, always on, clear() to free memory), L2 cache (shared across sessions, @Cacheable/@Cache on entity, providers: Ehcache/Hazelcast/Infinispan), cache concurrency strategies (READ_ONLY / NONSTRICT_READ_WRITE / READ_WRITE / TRANSACTIONAL), query cache (careful with invalidation)
4.7	Optimistic & Pessimistic Locking	@Version (numeric/timestamp, incremented on UPDATE, OptimisticLockException on conflict), LockModeType.OPTIMISTIC_FORCE_INCREMENT, LockModeType.PESSIMISTIC_READ/WRITE/FORCE_INCREMENT (SELECT FOR UPDATE), lock timeout hint (javax.persistence.lock.timeout), deadlock handling, when to use each
4.8	Dirty Checking & Flush Modes	Hibernate dirty checking: snapshot of entity state at load time, compared at flush, generates minimal UPDATE. FlushMode: AUTO (before query + before commit), COMMIT (only before commit), MANUAL (explicit flush), ALWAYS. @DynamicUpdate (only changed columns), @DynamicInsert, @Immutable (read-only entity, no dirty check). Open-session-in-view anti-pattern
4.9	Database Migrations – Flyway & Liquibase	Flyway: versioned migrations (V1__init.sql), repeatable (R__views.sql), undo (pro), flyway_schema_history table, maven/gradle plugin, spring.flyway.* config. Liquibase: changeSet (id+author), changelog formats (XML/YAML/JSON/SQL), rollback, databaseChangeLog, liquibase.properties. Best practices: never edit applied migrations, pre-deploy vs post-deploy
4.10	JPA Performance Tuning	Batch inserts: hibernate.jdbc.batch_size=50 + spring.jpa.properties.hibernate.order_inserts=true. JDBC fetch size (setFetchSize). StatelessSession for ETL. Read-only transactions (readOnly=true → no dirty check, no snapshot). DTO projections (SELECT new com.app.UserDTO(u.name,u.email) FROM User u). Query plan cache. HikariCP tuning (maximumPoolSize, connectionTimeout)

Chapter 5 SQL Databases & Query Optimization (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
5.1	SQL Fundamentals & DDL/DML	DDL (CREATE/ALTER/DROP/TRUNCATE), DML (INSERT/UPDATE/DELETE/MERGE/UPSERT), DCL (GRANT/REVOKE), TCL (COMMIT/ROLLBACK/SAVEPOINT). Data types (INT/BIGINT/VARCHAR/TEXT/TIMESTAMP/JSONB/UUID). Constraints: PK/FK (ON DELETE CASCADE/SET NULL)/UNIQUE/CHECK/NOT NULL/DEFAULT
5.2	Indexes – Types & Internals	B-Tree (balanced tree, $O(\log n)$, range queries, default type), Hash (equality only, $O(1)$), Composite (column order: most selective first, leftmost-prefix rule), Covering index (index-only scan, all queried columns in index), Partial index (WHERE condition), Functional index (LOWER(email)), index bloat & REINDEX
5.3	Query Optimization & EXPLAIN	EXPLAIN ANALYZE output: cost/actual time/rows/loops, Seq Scan vs Index Scan vs Index-Only Scan vs Bitmap Heap Scan, nested loop vs hash join vs merge join, planner statistics (pg_stats, ANALYZE), pg_stat_statements (top slow queries), auto-explain, query hints (enable_seqscan=off), parameterized queries prevent re-plan
5.4	Transactions & ACID	Atomicity (WAL redo log, all-or-nothing), Consistency (constraints enforced during tx), Isolation (concurrent tx separation – see isolation levels), Durability (fsync, WAL flushed to disk on commit). Savepoints (partial rollback). Long transactions (lock bloat, vacuum blocking). XA distributed transactions
5.5	Isolation Levels & MVCC	READ UNCOMMITTED / READ COMMITTED (default PG) / REPEATABLE READ / SERIALIZABLE. Anomalies: dirty read, non-repeatable read, phantom read, write skew. MVCC in PostgreSQL: each row version has xmin/xmax, snapshot taken at tx start, old versions cleaned by VACUUM. Serializable Snapshot Isolation (SSI) in PG 9.1+
5.6	Advanced SQL – Joins & Window Functions	INNER/LEFT/RIGHT/FULL OUTER/CROSS/SELF JOIN, correlated subqueries, CTEs (WITH clause, recursive CTE for hierarchies/graphs), window functions: ROW_NUMBER()/RANK()/DENSE_RANK()/NTILE()/LAG()/LEAD()/FIRST_VALUE()/LAST_VALUE() with PARTITION BY/ORDER BY/ROWS BETWEEN frame
5.7	Stored Procedures, Functions & Triggers	PL/pgSQL: procedures (CALL, can COMMIT inside) vs functions (SELECT, must return), variables/loops/cursors/exception handling. Triggers: BEFORE/AFTER/INSTEAD OF, FOR EACH ROW/STATEMENT, NEW/OLD records. Trigger use cases: audit log, soft delete, denormalized counter. Anti-patterns: hiding business logic in DB
5.8	Table Partitioning	Range partitioning (date ranges, monthly logs), List partitioning (region/status), Hash partitioning (even distribution). Partition pruning (query planner skips irrelevant partitions). Partition-wise joins. Default partition. Adding partitions without locking. pg_partman for automated partition management. MySQL partitioning
5.9	Replication & High Availability	Streaming replication (WAL sender/receiver, primary → standby), replication lag monitoring (pg_stat_replication), logical replication (table-level, cross-version, publication/subscription). Failover: Patroni (etcd/Consul/ZooKeeper leader election), pgpool-II. Connection pooling: PgBouncer (session/transaction/statement mode). Split-brain: STONITH fencing
5.10	PostgreSQL vs MySQL – Key Differences	MVCC: PG uses tuple versioning (multiple row versions in heap), MySQL InnoDB uses undo log. Full-text search: PG (tsvector/tsquery, GIN index) vs MySQL (FULLTEXT index). JSON: PG jsonb (binary, indexable, operators) vs MySQL JSON. CTEs: PG materialised by default, MySQL 8+ CTEs. Extensions: PostGIS, pg_trgm, pgvector, uuid-oss

Chapter 6 NoSQL Databases (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
6.1	NoSQL Types & CAP Theorem	Document (MongoDB), Wide-Column (Cassandra/HBase), Key-Value (Redis/DynamoDB), Graph (Neo4j), Time-Series (InfluxDB/TimescaleDB). CAP: choose 2 of 3 (CP: HBase/ZooKeeper, AP: Cassandra/DynamoDB). BASE (Basically Available, Soft state, Eventually consistent) vs ACID. Polyglot persistence

No.	Sub-Chapter Title	Key Topics Covered
6.2	Redis – Data Structures & Patterns	String (GET/SET/INCR/EXPIRE), List (LPUSH/RPOP, message queue), Hash (HSET/HGETALL, object store), Set (SADD/SMEMBERS/SINTER), Sorted Set (ZADD/ZRANGE with score, leaderboard), Stream (XADD/XREAD, event log). Pub/Sub, Lua scripting (atomic), Redis Cluster (hash slots 0-16383), Sentinel (HA), persistence (RDB snapshot vs AOF append-only)
6.3	Cassandra – Wide-Column Store	Partition key (data distribution, query MUST include), clustering key (sort order within partition), token ring + virtual nodes (vnodes), replication factor (RF=3 recommended), consistency levels (ONE/QUORUM/ALL/LOCAL_QUORUM), read/write path (memtable→SSTable→compaction). Anti-patterns: unbounded partitions, ALLOW FILTERING, secondary index over-use
6.4	DynamoDB – AWS Managed NoSQL	Table design: partition key + optional sort key, item = document (flexible schema). GSI (Global Secondary Index – different PK, async replication, eventual consistent reads), LSI (Local Secondary Index – same PK, different SK, strongly consistent). Single-table design pattern. DynamoDB Streams + Lambda triggers. DAX (in-memory cache). On-demand vs provisioned capacity + auto-scaling
6.5	MongoDB – Document Store	BSON document model, collections, embedded vs referenced documents. Indexes: single/compound/text/geospatial/TTL/partial. Aggregation pipeline: \$match/\$group/\$lookup/\$project/\$unwind/\$facet/\$graphLookup. Replica sets (primary + secondaries, oplog, automatic failover). Sharding (shard key choice critical). Atlas Search (Lucene-based full-text)
6.6	Graph Databases – Neo4j	Nodes (entities), Relationships (directed, typed, with properties), Labels (node type), Properties (key-value on nodes/relationships). Cypher: MATCH (n:Person)-[:KNOWS]->(m) WHERE n.name='Alice' RETURN m. Use cases: social networks (friends-of-friends), fraud detection (ring transactions), recommendation engines, dependency graphs, knowledge graphs
6.7	Time-Series Databases	InfluxDB: measurements (table), tags (indexed metadata), fields (values), time. Flux query language, continuous queries, retention policies. TimescaleDB: hypertable (PostgreSQL extension, automatic time-based partitioning, full SQL support). ClickHouse: columnar storage, vectorised execution, petabyte scale OLAP, Kafka integration
6.8	NoSQL Data Modeling Patterns	Embed vs reference (document DB): embed for data accessed together, reference for independently queried data. Denormalization for query patterns (pre-join data). Fan-out on write vs read. Bucket pattern (group many small documents). Schema versioning strategies. Handling schema changes without downtime (multi-version support in code)
6.9	Choosing SQL vs NoSQL	SQL: structured data, complex queries, ACID transactions, reporting. NoSQL: flexible schema, horizontal scale, high write throughput, simple access patterns. NewSQL: CockroachDB (distributed SQL, SERIALIZABLE), Google Spanner (globally distributed, TrueTime). Decision framework: query patterns, consistency needs, team expertise, operational cost
6.10	Cloud-Native & Multi-Model Databases	Azure Cosmos DB (multi-model: SQL/Mongo/Cassandra/Gremlin/Table APIs, turnkey global distribution, 5 consistency levels). FaunaDB (serverless, globally distributed, Calvin protocol). ArangoDB (multi-model: document+graph+key-value, AQL). Managed DB comparison: RDS vs Aurora (5x faster, storage auto-scale) vs Cloud SQL vs AlloyDB

Chapter 7 Build Tools – Maven & Gradle (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
7.1	Maven Build Lifecycle & POM	Three lifecycles: default (validate→compile→test→package→verify→install→deploy), clean (pre-clean→clean→post-clean), site. Effective POM (inheritance from super POM + parent). Multi-module reactor (dependency-sorted build order). -pl (project list), -am (also make), -amd (also make dependents)
7.2	Maven Dependency Management	Dependency scopes: compile (default, all classpaths), provided (compile+test, not runtime – servlet-api), runtime (not compile – JDBC driver), test (test only – JUnit), system (local JAR), import (BOM in dependencyManagement). Transitive deps, mediation (nearest-wins), exclusions, BOM (import scope), to standardise versions

No.	Sub-Chapter Title	Key Topics Covered
7.3	Maven Plugins	Surefire (unit tests, parallel execution, @Category filtering), Failsafe (integration tests, pre/post-integration-test phases), Shade (fat JAR, class relocation to avoid conflicts), Assembly (custom distributions), Compiler (source/target/release), Exec (run Java main), Release (tag + version bump), Enforcer (min Java/Maven version rules)
7.4	Maven Multi-Module Projects	Parent POM (packaging=pom, aggregator + parent), child modules inherit dependencies/plugins, reactor build order (dependency graph), section, Maven wrapper (mvnw ensures consistent version), CI-friendly versions (\${revision}+\${changelist} with flatten-maven-plugin), partial builds with -pl
7.5	Gradle Build Script & Tasks	Groovy DSL vs Kotlin DSL (build.gradle vs build.gradle.kts, type-safe), Task lifecycle: initialization → configuration (all tasks configured) → execution (only needed tasks run). Task definition: tasks.register vs tasks.create (lazy vs eager). Gradle wrapper (gradlew). settings.gradle (project name, included builds)
7.6	Gradle Dependency Management	Configurations: implementation (compile+runtime, not leaked to consumers), api (leaked – use sparingly), compileOnly (annotations, not runtime), runtimeOnly (JDBC driver), testImplementation. Version catalogs (gradle/libs.versions.toml, typesafe accessors). Dependency constraints. Platform (enforced-platform for BOM). Resolution strategies (force, reject)
7.7	Gradle Multi-Module & Build Cache	Multi-project build (settings.gradle: include 'lib','app'), project dependencies (implementation(project(':lib'))), allprojects/subprojects/configure(subprojects.filter{...}). Build cache: local (~/.gradle/caches) + remote (Gradle Enterprise, S3). Configuration cache (serialize task graph). Task avoidance (register, not create). Incremental compilation
7.8	Gradle Plugins	Java plugin (compileJava/test/jar tasks), Spring Boot plugin (bootJar, bootRun, layered JARs for Docker), Shadow plugin (fat JAR with relocation), OWASP dependency-check, JaCoCo (code coverage, minimum threshold enforcement), Checkstyle/PMD/SpotBugs (static analysis), custom plugin (buildSrc or standalone, extension DSL)
7.9	Maven vs Gradle – Deep Comparison	Performance: Gradle wins (incremental builds, build cache, parallel execution, configuration cache). Flexibility: Gradle (Groovy/Kotlin DSL, any logic) vs Maven (XML, convention over configuration, plugins only). Learning curve: Maven easier to start, Gradle more powerful. IDE support: both excellent (IntelliJ/Eclipse). Migration: maven2gradle script + manual adjustments
7.10	Build Tool CI/CD Integration	GitHub Actions: cache ~/.m2 or ~/.gradle, matrix builds (Java 17/21), reusable workflows. Publishing: Nexus/Artifactory (maven-publish/publish plugins, SNAPSHOT vs RELEASE). GitHub Packages. Docker image in pipeline (Jib plugin – no Dockerfile needed, buildpacks). SBOM generation (CycloneDX maven/gradle plugin). Semantic versioning (git-semver, release-please)

Phase 3: Application Security

Chapter 8 Application Security (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
8.1	TLS/HTTPS & X.509 Certificates	TLS 1.3 handshake (ClientHello→ServerHello→Certificate→CertificateVerify→Finished), symmetric vs asymmetric encryption, X.509 fields (CN/SAN/validity/issuer/serial/keyUsage), CA chain (root→intermediate→leaf), certificate pinning, mTLS (mutual TLS, client certificate), OCSP stapling, certificate transparency (CT logs)
8.2	Authentication – Sessions, Cookies & JWT	Session-based (server-side store, session ID in cookie), cookie attributes (HttpOnly/Secure/SameSite=Strict/Lax/Domain/Path/Max-Age). Stateless JWT (header.payload.signature, HMAC-SHA256 or RS256), JWT vulnerabilities (alg:none attack, weak HMAC secret, no expiry). Refresh token rotation, token storage: httpOnly cookie (XSS safe) vs localStorage (CSRF safe)
8.3	OAuth 2.0 Flows	Authorization Code + PKCE (web/mobile apps, code_verifier/code_challenge SHA256), Client Credentials (service-to-service, no user), Implicit (deprecated – token in URL fragment), ROPC (deprecated). Scopes (fine-grained permissions), access token (short-lived, opaque or JWT), refresh token (long-lived, rotation), token introspection endpoint, resource server validation
8.4	OpenID Connect (OIDC) & SSO	OIDC = OAuth2 + identity (ID token is JWT with sub/email/name claims), UserInfo endpoint, discovery document (/.well-known/openid-configuration), SSO: one login across multiple apps via shared IdP session. OIDC flows (code+PKCE preferred). Keycloak/Auth0/Okta integration (Spring Security oauth2Login). Session management: RP-initiated logout, front-channel/back-channel logout
8.5	SAML 2.0 & Enterprise SSO	SAML assertions (authentication/attribute/authorization decision), SP-initiated vs IdP-initiated SSO flows, metadata XML exchange, assertion signing (RSA) + encryption (AES), Spring Security SAML2 configuration, federation with Active Directory/LDAP (ADFS), SAML vs OIDC (enterprise vs modern web), attribute mapping
8.6	Spring Security Architecture	SecurityFilterChain (ordered filter list), DelegatingFilterProxy→FilterChainProxy, authentication flow: UsernamePasswordAuthenticationFilter→AuthenticationManager→ProviderManager→DaoAuthenticationProvider→UserDetailsService. SecurityContext (ThreadLocal, cleared after request). @PreAuthorize('hasRole("ADMIN")' and #id==authentication.principal.id'), SpEL expressions, method security proxy
8.7	API Security	API key (header X-API-Key, hash in DB, rate-limit per key), request signing (HMAC-SHA256 of body+timestamp, replay attack prevention via nonce/timestamp window), rate limiting: token bucket algorithm (Bucket4j + Redis), Redis sliding window log. mTLS for service-to-service. API gateway security (Kong rate-limit/key-auth plugins, AWS WAF + API Gateway)
8.8	OWASP Top 10 & Injection Prevention	SQL injection: PreparedStatement/parameterized queries (never string concat). XSS: output encoding (OWASP Java Encoder), Content-Security-Policy header. CSRF: SameSite=Strict cookie + CSRF token (Spring Security CsrfFilter). IDOR: always authorise resource ownership. XXE: disable external entity parsing. Insecure deserialization: whitelist classes. SSRF: allowlist outbound URLs
8.9	Password Hashing & Cryptography	Never store plaintext. Adaptive hashing: BCrypt (cost factor 10-12, 60-char output), SCrypt (memory-hard), Argon2id (winner PHC, recommended). PBKDF2 (FIPS compliant). Salting (auto with BCrypt). Spring Security PasswordEncoder (BCryptPasswordEncoder). AES-GCM (authenticated symmetric encryption). RSA-OAEP/ECDSA for asymmetric. AWS Secrets Manager / HashiCorp Vault
8.10	RBAC, ABAC & Zero Trust	RBAC: roles → permissions → resources (User has ROLE_ADMIN grants all /admin/** endpoints). ABAC: policy = subject attributes + resource attributes + environment (time/location), XACML/OPA (Open Policy Agent) as policy engine. Spring Security @PreAuthorize with SpEL for both. Zero Trust: verify every request (no implicit trust), device posture check, mTLS between services, service mesh (Istio authz policies)

Phase 4: Application Capabilities

Chapter 9 Caching – All Levels (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
9.1	CPU Caches & Memory Hierarchy	L1 (~4KB, ~1ns, per core), L2 (~256KB, ~4ns, per core), L3 (~8MB, ~10ns, shared), RAM (~64ns), SSD (~100µs). Cache line = 64 bytes. False sharing: two threads write adjacent fields on same cache line → cache invalidation storm. Padding/@Contended to fix. NUMA: local vs remote memory access, thread affinity
9.2	JVM Object & Heap Caching	Eden gen as implicit cache for short-lived objects. Object pooling (Apache Commons Pool, ByteBuffer pool for NIO). String interning (String.intern() – use cautiously, Metaspace overhead). Flyweight pattern in JVM (Integer.valueOf cache). Off-heap memory (DirectByteBuffer, Chronicle Map, Unsafe) – GC-free large caches
9.3	Application-Level Caching	Spring Cache abstraction (@Cacheable(value='users',key='#id'), @CacheEvict(allEntries=true), @CachePut, @Caching for multi-op). Caffeine (W-TinyLFU eviction, best in-process cache, maximumSize/expireAfterWrite/refreshAfterWrite). Guava LoadingCache (older, fewer features). Cache stampede prevention: probabilistic early expiration, request coalescing (single-flight), distributed lock on cache miss
9.4	Distributed Cache – Redis	Cache-aside with Spring: @Cacheable maps to Redis GET/SET/EXPIRE. Redis data types for caching: String (simple values), Hash (partial updates), Sorted Set (time-ranked). Redis Cluster (16384 hash slots, MOVED/ASK redirects). Redisson (Java client, RMap/RBucket, distributed lock RLock, rate limiter, publish/subscribe). Cache serialization (JSON vs Kryo vs Protobuf performance)
9.5	Distributed Cache – Hazelcast & Memcached	Hazelcast: IMap distributed across cluster members (partitioned, 271 partitions by default), near-cache (local copy, invalidation on update), IMap EntryListener, CP subsystem (Raft-based for strong consistency). Memcached: simple key-value, multi-threaded, no persistence/replication, consistent hashing client-side. When to choose: Redis (rich features) vs Memcached (simplicity, raw throughput)
9.6	Database Query Cache & Connection Pooling	JPA L2 query cache (invalidated on any entity change – be careful). MySQL query cache (removed in 8.0 – too coarse). HikariCP: maximumPoolSize (CPU*2+disk spindles), minimumIdle, connectionTimeout (30s default), idleTimeout, maxLifetime (< DB wait_timeout), leakDetectionThreshold. PgBouncer: session (full connection) / transaction (recommended) / statement pooling mode
9.7	HTTP Caching	Cache-Control: max-age=3600 (client cache TTL), s-maxage (CDN TTL), no-cache (revalidate before use), no-store (never cache), private (browser only), public (CDN allowed), must-revalidate. ETag: strong (byte-for-byte identical) vs weak (W/"..."). Conditional GET: If-None-Match/If-Modified-Since → 304 Not Modified. Vary header (vary by Accept-Encoding/Accept-Language)
9.8	CDN & Edge Caching	CloudFront: distributions, origins (S3/ALB/custom), behaviors (path patterns, cache policies, origin request policies), TTL (min/default/max), invalidation (path or wildcard, 1000 free/month), Lambda@Edge/CloudFront Functions (request/response manipulation at edge). Cloudflare: cache rules, page rules, Cache API in Workers. Fastly: instant purge (150ms SLA), VCL customization, shielding
9.9	Cache Strategies	Cache-aside (lazy loading, app checks cache→miss→load DB→populate cache; risk: stale data, stampede). Read-through (cache loads from DB on miss, transparent to app). Write-through (write to cache+DB synchronously, always consistent, higher write latency). Write-behind/write-back (write to cache, async flush to DB, risk: data loss). Refresh-ahead (proactively refresh before TTL expires)

No.	Sub-Chapter Title	Key Topics Covered
9.10	Cache Eviction Policies & Consistency	LRU (Least Recently Used – O(1) with doubly-linked list + HashMap, Caffeine). LFU (Least Frequently Used, tracks frequency, Caffeine TinyLFU). TTL (time-based, simple). ARC (Adaptive Replacement Cache, balances recency and frequency). Cache invalidation: event-driven (message bus on data change), write-through, polling. Distributed cache coherence: MESI protocol analogy, eventual consistency

Chapter 10 Search – Elasticsearch & Alternatives (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
10.1	Search Fundamentals & Inverted Index	Inverted index: term dictionary maps each term → posting list (doc IDs + positions + TF). Tokenization (split on whitespace/punctuation), normalization (lowercase, ASCII folding), stemming (running→run), stop words removal. Why SQL LIKE '%query%' doesn't scale (no index usage). Forward index vs inverted index trade-off
10.2	Elasticsearch Architecture	Cluster (multiple nodes) → Node (master-eligible/data/coordinating/ingest) → Index → Shard (Lucene index) → Document. Primary + replica shards (HA + read scale). Shard routing: <code>hash(_id) % num_primary_shards</code> . Split-brain prevention: <code>minimum_master_nodes</code> (< 7.x), voting configuration (7.x+, odd number). Coordinating node: fan-out to shards, reduce/merge results
10.3	Index Mappings & Analyzers	Explicit mapping (prevent dynamic mapping surprises): text (full-text, analyzed), keyword (exact match, sorting, aggregations), date (epoch_millis/strict_date_optional_time), integer/float, nested (array of objects with independent queries), object. Custom analyzer: <code>char_filter</code> (HTML strip) + <code>tokenizer</code> (standard/whitespace/ngram) + <code>token_filter</code> (lowercase/stop/synonym/stemmer). Index templates + component templates
10.4	Elasticsearch Query DSL	Match (full-text, analyzed), multi_match (multiple fields, best_fields/most_fields/cross_fields), match_phrase (exact phrase, slop parameter). Bool: must (AND, affects score) / should (OR, boosts score) / must_not (AND NOT) / filter (AND, no score, cached). Term/terms/range/exists/prefix/wildcard/regexp. Nested query for nested type. Aggregations: terms/date_histogram/avg/min/max/sum/cardinality + bucket+pipeline aggs
10.5	Relevance Scoring & Boosting	BM25 (default): $IDF(\text{rare terms score higher}) * TF(\text{more occurrences score higher}) * \text{field_length_norm}$. b parameter (field length normalisation, 0=no norm, 1=full norm), k1 (TF saturation). Field boosting (^2 in query). <code>function_score</code> : <code>field_value_factor</code> (multiply score by field value), <code>gauss/linear/exp</code> decay (geospatial/date relevance decay), <code>random_score</code> (randomised results). Script score. Pinned queries (elevate specific docs)
10.6	Autocomplete & Fuzzy Search	Completion suggester (FST – Finite State Transducer, sub-millisecond, prefix only). <code>edge_ngram</code> analyzer (index 'java' as 'j','ja','jav','java' – supports prefix search with match query). <code>search_as_you_type</code> field type (shingles, root+2gram+3gram). Fuzzy query (Levenshtein distance, fuzziness:AUTO – 0 for 1-2 chars, 1 for 3-5, 2 for 6+). Phonetic analysis (Soundex/Metaphone for name matching). Suggest API (term/phrase suggesters for spell correction)
10.7	Spring Data Elasticsearch	@Document(indexName='users'), @Field(type=FieldType.Text/Keyword/Date), @Id. ElasticsearchRepository (findByName, custom @Query). ElasticsearchOperations (NativeQueryBuilder, SearchHits result). ReactiveElasticsearchClient (WebFlux). Index Lifecycle Management (ILM): hot→warm→cold→frozen→delete phases. Index templates, aliases (zero-downtime reindex), rollover (new index when size/age threshold met)
10.8	Search Data Sync – CDC & Debezium	Problem: DB is source of truth, ES is search replica – how to keep in sync? Options: (1) Dual-write (risk: partial failure), (2) Application event + Kafka, (3) CDC via Debezium (PostgreSQL logical replication slot → Kafka → Kafka Connect Elasticsearch Sink Connector). Debezium captures INSERT/UPDATE/DELETE as Kafka messages. Bulk indexing via <code>_bulk</code> API (batch of index/update/delete actions)

No.	Sub-Chapter Title	Key Topics Covered
10.9	Elasticsearch Performance Tuning	Shard sizing: 10-50 GB per shard (too small = overhead, too large = slow recovery). <code>refresh_interval</code> : default 1s (increase to 30s during bulk indexing). <code>translog.durability</code> : request (default, safe) vs async (faster, risk). <code>force_merge</code> to 1 segment for read-only indices. <code>doc_values</code> (columnar, for sort/agg, default on keyword) vs <code>fielddata</code> (expensive, avoid on text). JVM heap: 50% of RAM, never exceed 32 GB (compressed OOPs limit)
10.10	Search Alternatives	OpenSearch: ES 7.10 fork by AWS, fully open-source (Apache 2.0), managed on AWS. Apache Solr: mature, Lucene-based, ZooKeeper-based clustering (SolrCloud), strong faceting, good for enterprise search. Meilisearch: typo-tolerant out of box, fast (~50ms), simple API, ideal for SaaS product search, Rust-based. Typesense: open-source Algolia alternative, auto-typo-tolerance, instant search, easy ops. Algolia: SaaS, best-in-class DX, expensive at scale

Chapter 11 Concurrency & Locking (Application Level) (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
11.1	Optimistic Concurrency Control (OCC)	@Version field (int/long/Instant) in JPA entity, Hibernate increments on every UPDATE, includes WHERE version=? check. OptimisticLockException on conflict → catch + retry (3 attempts with backoff). Conditional UPDATE at JDBC level (UPDATE SET ... WHERE id=? AND version=?). Best for: low-conflict, read-heavy, short transactions. Compare to database-level MVCC
11.2	Pessimistic Locking	SELECT FOR UPDATE (PostgreSQL/MySQL/Oracle row-level lock, held until tx commit/rollback). JPA LockModeType.PESSIMISTIC_WRITE. Lock timeout: SET LOCAL lock_timeout='3s' or javax.persistence.lock.timeout hint. SKIP LOCKED: skip already-locked rows (job queue pattern, fair work distribution). Deadlock risk: always lock in consistent order. Table lock vs row lock vs page lock
11.3	ETags & HTTP Conditional Requests	ETag: server generates fingerprint (MD5/SHA of content or version number). Client stores ETag from GET response. On update: PUT/PATCH/DELETE with If-Match: "etag-value" header. Server: compare If-Match vs current ETag → match: proceed → no match: 412 Precondition Failed (someone else changed it). If-None-Match on GET → 304 Not Modified (caching + optimistic check). Implementation in Spring (@RequestHeader If-Match, ShallowEtagHeaderFilter)
11.4	Distributed Locks	Redis SETNX+EXPIRE (atomic via SET key value NX EX seconds). Redlock algorithm (majority of N Redis nodes, criticised by Martin Kleppmann – use fencing tokens). ZooKeeper ephemeral sequential nodes (leader election, fair lock). PostgreSQL advisory locks: pg_try_advisory_lock(key) (non-blocking), pg_advisory_lock(key) (blocking). Fencing token: monotonically increasing number per lock acquisition, storage rejects old tokens
11.5	Two-Phase Locking & Deadlocks	2PL: growing phase (acquire locks), shrinking phase (release locks, no new acquisitions). Strict 2PL: release all locks at commit (prevents cascading aborts). Deadlock: circular wait (A waits B, B waits A). Detection: wait-for graph (PostgreSQL auto-detects, kills one tx). Prevention: lock ordering (always lock lower ID first), lock timeout (deadlock_timeout = 1s in PG). Application deadlocks: JDBC batch order matters
11.6	MVCC & Snapshot Isolation	MVCC: each tx sees consistent snapshot of DB at tx start (no read locks needed). PostgreSQL: row has xmin (created by tx) + xmax (deleted by tx), snapshot = list of active tx at start. Write skew anomaly: two txs read same data, both decide to write based on stale read (e.g., on-call scheduling). Fix: SELECT FOR UPDATE or SERIALIZABLE isolation. Serialization failures: SQLSTATE 40001, retry loop
11.7	Application-Level Conflict Resolution	409 Conflict response: include current version + conflicting resource in body. User-facing: 'Someone else modified this while you were editing. Please review the changes.' Merge strategies: last-write-wins (simple, data loss risk), 3-way merge (original + A's changes + B's changes), field-level merge (merge non-conflicting fields automatically). Booking hold: reserve seat for 15 min (status=HELD), timer releases if not confirmed

No.	Sub-Chapter Title	Key Topics Covered
11.8	Event Sourcing & Idempotency	Event sourcing: store sequence of domain events (OrderPlaced/OrderShipped) not current state → rebuild state by replaying. Idempotency keys: client sends UUID with request, server stores processed keys → duplicate request returns stored response without re-executing. Exactly-once: Kafka transactions (atomic produce + consumer offset commit). Outbox pattern: write event to outbox table in same DB tx, relay reads outbox + publishes to Kafka
11.9	CRDTs & Collaborative Editing	CRDT (Conflict-Free Replicated Data Type): data structure that auto-merges concurrent edits without conflicts. G-Counter (grow-only, per-node counters, merge=max per node). LWW-Register (Last-Write-Wins, timestamp-based). OR-Set (add-wins, unique tags). Operational Transformation (OT): transform concurrent ops to be compatible (Google Docs). Libraries: Yjs (CRDT for text/JSON, real-time collab), Automerge
11.10	Database Lock Monitoring	PostgreSQL: pg_locks (relation/tuple/advisory locks, granted vs waiting), pg_stat_activity (query, state, wait_event_type). Find blocking: SELECT * FROM pg_stat_activity WHERE wait_event_type='Lock'. kill blocker: SELECT pg_terminate_backend(pid). lock_timeout + statement_timeout (set per session). MySQL InnoDB: INFORMATION_SCHEMA.INNODB_TRX/INNODB_LOCK_WAITS. Deadlock log (innodb_print_all_deadlocks=ON)

Chapter 12 i18n – Multi-Language, Currency & Timezone (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
12.1	Internationalization Fundamentals	i18n (internationalization – making software adaptable), L10n (localization – adapting for specific locale), g11n (globalization = i18n + L10n). Locale: language (ISO 639 'en') + country (ISO 3166 'US') + variant. Unicode: universal character set (1.1M+ code points), UTF-8 encoding (1-4 bytes, backward compatible with ASCII). ICU (International Components for Unicode) library. CLDR (Common Locale Data Repository)
12.2	Message Bundles & Resource Files	Java ResourceBundle: messages_en.properties, messages_fr_FR.properties, messages.properties (fallback). Spring MessageSource: ReloadableResourceBundleMessageSource (file-based, hot-reload), ResourceBundleMessageSource (classpath JAR). MessageFormat patterns: {0,number,currency} {1,date,short}. ICU4J MessageFormat for advanced pluralization (# items, 1 item). Missing key strategy: throw vs fallback to key
12.3	Number & Currency Formatting	NumberFormat.getCurrencyInstance(Locale.FRANCE) → '1 234,56 €'. DecimalFormat patterns ('#,##0.00'). JSR 354 JavaMoney (javax.money): Money.of(BigDecimal, "EUR"), MonetaryAmounts, ExchangeRateProvider (ECB rates). Rounding: RoundingMode.HALF_EVEN (banker's rounding) for financial calculations. Store money as: long cents (no floating point) or BigDecimal (exact). Never use double for money
12.4	Date & Time Localization	Java 8 java.time: LocalDate/LocalTime/LocalDateTime (no timezone), ZonedDateTime (with zone), Instant (epoch seconds, for storage/comparison). DateTimeFormatter.ofPattern('dd/MM/yyyy', Locale.UK). ZonedDateTime.of('America/New_York') vs ZoneOffset.of('+05:30'). DST transitions: ZonedDateTime handles automatically. User timezone stored as IANA ID string (not offset – offsets change with DST)
12.5	Timezone Handling in APIs & Databases	Golden rule: store UTC in database (TIMESTAMP WITH TIME ZONE in PostgreSQL converts to UTC), display in user's timezone. API: accept ISO 8601 with offset (2024-01-15T14:30:00+05:30), store as Instant. Response: include UTC Instant + user's local formatted string. PostgreSQL: AT TIME ZONE operator. MySQL: CONVERT_TZ(). Common pitfall: TIMESTAMP in MySQL stores/retrieves in server timezone (use DATETIME + application-level conversion)
12.6	Character Encoding	UTF-8: 1 byte (ASCII 0-127), 2-3 bytes (most scripts), 4 bytes (emoji/CJK extension). UTF-16: Java's native char (2 bytes, BMP), supplementary chars need 2 chars (surrogate pair). BOM (Byte Order Mark): EF BB BF for UTF-8 (avoid), FE FF for UTF-16 BE. MySQL: utf8 (3-byte, no emoji) vs utf8mb4 (4-byte, required for emoji). HTTP: charset in Content-Type header. Java String: UTF-16 internally, use getBytes(StandardCharsets.UTF_8)

No.	Sub-Chapter Title	Key Topics Covered
12.7	RTL Language Support	RTL languages: Arabic, Hebrew, Farsi, Urdu. HTML: dir='rtl' on or . CSS: logical properties (margin-inline-start instead of margin-left, padding-block-end instead of padding-bottom) for automatic RTL flipping. Bidirectional text (BiDi): Unicode BiDi algorithm handles mixed LTR+RTL. Font: Google Fonts Noto family supports most scripts. Dates/numbers: still LTR within RTL text. Testing: Chrome DevTools 'Emulate RTL'
12.8	Multi-Tenancy & Locale Per User	Store locale preference: users table column locale VARCHAR(20) (e.g., 'en-US', 'de-DE'). Tenant-level defaults (organisation sets default locale for all users). Request locale resolution chain (Spring LocaleContextHolder): URL path (/en/products) → URL param (?lang=fr) → Accept-Language header negotiation → user preference from DB → system default. LocaleChangeInterceptor for param-based switching. Thread-local locale storage
12.9	Translation Workflow & Management	Externalize ALL user-facing strings (no hardcoded text in code). Pseudo-localization: replace chars with accented variants + add 40% length padding to catch truncation issues before translators. Translation Management Systems: Phrase (formerly Phrase), Lokalise, Crowdin – integrate with CI/CD, machine pre-translation (DeepL/Google), human review workflow. Missing key monitoring in production. Pluralization testing (1 item vs 2 items vs 0 items)
12.10	Testing i18n Applications	Test matrix: run test suite with en/fr/de/ar/ja locales. Pseudo-locale (en-XA): automatically expanded text to find layout breaks. Boundary cases: longest translated string (German compound words: Donaudampfschiffahrtsgesellschaft), CJK characters (fixed-width), RTL text direction. Number/date format regression tests. Timezone edge cases: DST transition dates, UTC midnight crossing. Spring @LocaleContextHolder in tests

Chapter 13 Software Performance – Sync, Async & Reactive (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
13.1	Synchronous vs Asynchronous vs Semi-Sync	Synchronous: blocking call, thread waits for response (simple, debuggable, scales poorly for IO-bound). Asynchronous: non-blocking, callback/future/event notifies completion (complex, harder to debug, scales well). Semi-sync (async with blocking wait): CompletableFuture.get() – retains async benefits for concurrency but blocks thread. Amdahl's law: speedup limited by serial fraction. Choose based on IO/CPU ratio
13.2	Reactive Programming – Project Reactor	Reactive Streams spec: Publisher/Subscriber/Subscription/Processor. Mono (0-1 elements), Flux (0-N elements). Operators: map (sync transform), flatMap (async 1-to-many), concatMap (ordered flatMap), zip (combine two publishers), merge (concurrent), filter, take, skip, buffer, window, retry, onErrorResume. Schedulers: Schedulers.parallel() (CPU-bound), Schedulers.boundedElastic() (IO-bound blocking), immediate/single
13.3	Spring WebFlux & Non-Blocking IO	WebFlux vs Spring MVC: both support annotations, WebFlux uses Netty event loop (non-blocking). @RestController with Mono>/Flux return types. WebClient (non-blocking HTTP client, replaces RestTemplate). R2DBC (reactive DB drivers for PostgreSQL/MySQL/H2). ServerSentEvents (Flux>). WebSocket (WebSocketHandler). Reactive security filter chain (SecurityWebFilterChain)
13.4	Backpressure & Flow Control	Backpressure: consumer signals to producer how many items it can handle (Subscription.request(n)). Reactor strategies on overflow: buffer (unbounded – OOM risk), drop (discard excess), latest (keep newest), error (throw OverflowException). Bounded queues (Flux.onBackpressureBuffer(1000)). Kafka consumer: pause/resume partitions based on processing lag. TCP flow control analogy (receive window)
13.5	Thread Pool & Executor Tuning	ThreadPoolExecutor: corePoolSize (always alive), maxPoolSize (under load), queueCapacity (LinkedBlockingQueue=unbounded – dangerous, SynchronousQueue=no buffering), keepAliveTime, RejectedExecutionHandler (AbortPolicy throws, CallerRunsPolicy slows producer). ForkJoinPool (work-stealing, parallel streams). Virtual thread executor (Java 21, IO-bound services, replace fixed-size pools)

No.	Sub-Chapter Title	Key Topics Covered
13.6	Profiling & Performance Analysis	Java Flight Recorder (JFR): low-overhead (<1%) continuous profiling, built into JDK 11+. JDK Mission Control: analyse JFR recordings (CPU hotspot/allocation/GC/thread/IO). async-profiler: CPU (sampling via perf_events), allocation (TLAB sampling), lock profiling, wall-clock mode. Flame graphs: identify hot code paths visually. JVM safepoint pauses: check with -Xlog:safepoint. GC pause analysis: GCEasy.io
13.7	Load Testing	JMeter: thread groups (virtual users), HTTP samplers, assertions, listeners (Aggregate Report/Summary), distributed load (master+slaves), JMeter plugins (throughput shaping, PerfMon). Gatling: Scala DSL, realistic simulations (rampUsers/atOnceUsers/constantUsersPerSec), HTML report with p50/p95/p99/p999. k6: JavaScript, cloud execution (k6 Cloud), thresholds as SLOs, output to InfluxDB/Grafana. Metrics: throughput (RPS), latency percentiles, error rate
13.8	Memory Profiling & GC Tuning	Heap dump: jmap -dump:format=b,file=heap.hprof or -XX:+HeapDumpOnOutOfMemoryError. Eclipse MAT: Leak Suspects report, Dominator Tree (largest retained object graphs), OQL queries (SELECT * FROM java.util.ArrayList). Allocation profiling: async-profiler alloc mode, identifies hot allocation sites. Reducing GC pressure: object reuse (pooling), primitive arrays vs boxed, smaller objects, avoid finalizers. GC log analysis: -Xlog:gc*:file=gc.log:time,level
13.9	JMH Benchmarking	@Benchmark (mark benchmark method), @State(Scope.Thread/Benchmark/Group) (shared state), BenchmarkMode: Throughput (ops/s), AverageTime (s/op), SampleTime (distribution), SingleShotTime (cold start). @Warmup(iterations=5,time=1,timeUnit=SECONDS), @Measurement(iterations=10), @Fork(2). @Param (parameterised benchmarks). Blackhole.consume() to prevent dead code elimination. Pitfalls: JIT bias, constant folding, loop optimisation
13.10	Database Performance Tuning	Connection pool: HikariCP maximumPoolSize = (CPU*2)+disk_spindles (OLTP); monitor pool wait time (hikaricp.connections.pending). Slow query: pg_stat_statements (calls/total_time/mean_time), MySQL slow_query_log. Index coverage: EXPLAIN (Index Only Scan = no heap access). VACUUM/ANALYZE: prevent table bloat + update planner stats. Partitioning: prune partitions in WHERE clause. Read replicas: route SELECT to replica, reduce primary load. Async inserts for non-critical writes

Phase 5: Performance & Architecture

Chapter 14 System Design (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
14.1	System Design Methodology	Step 1: clarify requirements (functional: what system does; non-functional: scale/latency/availability/consistency). Step 2: capacity estimation ($DAU \times actions = QPS$, $storage = QPS \times payload \times retention$). Step 3: high-level design (components & data flow). Step 4: component deep-dive (bottleneck focus). Step 5: trade-off discussion. Back-of-envelope: 1M users, 10 req/user/day = 115 QPS average, 3x peak = 350 QPS
14.2	Scalability Patterns	Vertical scaling (bigger machine – simple, single point of failure, limit exists). Horizontal scaling (more machines – stateless services required, load balancer needed). Stateless app (session in Redis/DB, not in-process). Auto-scaling: target-tracking (CPU=70%), step scaling, scheduled. Predictive scaling (AWS, ML-based). Scale-to-zero: AWS Lambda, Cloud Run (cold start trade-off). Database scaling: read replicas, sharding
14.3	Consistent Hashing & Load Balancing	Consistent hashing: nodes placed on ring, key→nearest clockwise node, add/remove node redistributes only K/n keys (vs K keys in modulo). Virtual nodes (vnodes): each node placed multiple times for even distribution. Rendezvous (highest random weight) hashing. Load balancing: Round Robin, Least Connections, IP Hash (sticky), Least Response Time (NGINX Plus), Random. Health checks: active (ping endpoint) vs passive (monitor error rate)
14.4	CAP Theorem & Consistency Models	CAP: in presence of network partition, choose Consistency (all nodes see same data) or Availability (every request gets response). CA impossible at scale. CP: HBase, ZooKeeper, etcd. AP: Cassandra, DynamoDB, CouchDB. PACELC: extends CAP – even without partition, trade latency vs consistency. Consistency models: linearizable (strongest), sequential, causal, eventual (weakest). Dynamo-style: tunable consistency ($W+R>N$)
14.5	Message Queues & Event Streaming	Kafka: topic→partition (ordered within partition), offset (consumer position), consumer group (parallel consumers, 1 partition per consumer max), log compaction (keep latest per key), exactly-once (transactions + idempotent producer), schema registry (Avro/Protobuf). RabbitMQ: exchange (direct/topic/fanout/headers)→binding→queue, AMQP, DLX (dead-letter exchange), priority queues. SQS: standard (at-least-once) vs FIFO (exactly-once, ordered, 300TPS). Pulsar: multi-tenancy, tiered storage
14.6	API Gateway & Service Discovery	API gateway: single entry point, handles auth (JWT validation), rate limiting (token bucket), request routing, SSL termination, request/response transformation, load balancing, circuit breaking. Kong (plugin-based, open-source), AWS API Gateway (managed, Lambda integration), Nginx + Lua. Service discovery: client-side (Eureka, Consul – client queries registry, client load balances) vs server-side (ALB, Kubernetes Service DNS). Circuit breaker: CLOSED→OPEN (failures>threshold)→HALF-OPEN (probe)
14.7	Database at Scale – Sharding & CQRS	Read replicas: route SELECTs to replica, replication lag (eventual consistency). Sharding: partition data across DBs by shard key ($user_id \bmod N$, range, directory). Cross-shard queries: avoid or scatter-gather. Shard key choice (avoid hotspots). CQRS: Command (write, strong consistency, normalized DB) + Query (read, eventual consistency, denormalized read model). Sync via events/CDC. Distributed transactions: Saga (compensating txs) / 2PC (blocking) / TCC (try-confirm-cancel)
14.8	Observability – Metrics, Tracing & Logging	Metrics: Prometheus (pull-based scrape, PromQL, Alertmanager), Micrometer (instrumentation facade, Spring Boot Actuator), Grafana dashboards. Tracing: OpenTelemetry (standard SDK, OTLP protocol, auto-instrumentation Java agent), Jaeger (trace storage + UI), Zipkin. Trace context: traceId+spanId propagated via W3C traceparent header. Logging: structured JSON logs (logback-spring.xml), correlation ID (MDC.put('traceId',...)). Log aggregation: ELK (Elasticsearch+Logstash+Kibana) / Loki+Grafana

No.	Sub-Chapter Title	Key Topics Covered
14.9	System Design Walkthroughs – Part 1	URL Shortener: hash(longURL)→6-char code, redirect service (302 vs 301), counter-based (base62), collision handling, analytics (Kafka+ClickHouse). Rate Limiter: token bucket (Redis INCR+EXPIRE per user per window), sliding window log (Redis sorted set), sliding window counter (approximate, efficient). Web Crawler: BFS+frontier queue, politeness (robots.txt+delay), distributed (URL frontier sharding by domain hash), dedup (Bloom filter)
14.10	System Design Walkthroughs – Part 2	Twitter Feed: fan-out-on-write (push to followers' timelines on tweet, fast read, slow write for celebrities) vs fan-out-on-read (pull+merge at read time, slower read). Hybrid: push for regular users, pull for celebrities. Hotel/Ticket Booking: seat locking (status=HELD for 15min, cron cleanup), double-booking prevention (UNIQUE constraint + SELECT FOR UPDATE), payment idempotency. Uber: geospatial index (S2 cells/H3 hexagons), driver location updates (WebSocket), trip matching (proximity + filters)

Chapter 15 Microservices & Event-Driven Architecture (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
15.1	Monolith vs Microservices vs Modular Monolith	Monolith: single deployable, shared DB, simple ops, becomes big ball of mud. Microservices: independent deploy, separate DB per service, polyglot, operational complexity (network/distributed tracing/partial failure). Modular Monolith: clear module boundaries (Java 9 modules or package-private + ArchUnit enforcement), shared DB but module-owned tables, single deploy – best of both worlds for teams <50
15.2	Service Decomposition & Bounded Contexts	DDD Bounded Context: explicit model boundaries where terms have specific meaning. Context Map patterns: Shared Kernel (shared code/model), Customer-Supplier (upstream-downstream), Conformist (downstream conforms), Anti-Corruption Layer (translate between models), Open-Host Service (published protocol/API), Published Language (shared schema). Decompose by business capability (stable) not by technical layer (fragile)
15.3	REST vs gRPC vs GraphQL	REST: resource-based (/users/{id}), HTTP verbs (GET/POST/PUT/PATCH/DELETE), stateless, cacheable, HATEOAS (links in response), OpenAPI/Swagger. gRPC: Protocol Buffers (binary, 3-10x smaller than JSON), strongly typed, bidirectional streaming (HTTP/2), code generation (protoc), great for internal services, harder to debug. GraphQL: client specifies fields (no over/under-fetching), single endpoint, N+1 problem (DataLoader batching), schema-first, subscriptions via WebSocket
15.4	Event-Driven Architecture	Event types: notification (something happened, no data), event-carried state transfer (includes new state), domain event (business concept). Integration events (cross-service) vs domain events (within bounded context). Event schema: CloudEvents standard, Avro/Protobuf with schema registry (backward/forward compatibility). Choreography (services react to events, loose coupling, hard to trace) vs Orchestration (saga orchestrator directs flow, easy to trace, single point of coordination)
15.5	CQRS Pattern	Command side: handles writes, strong consistency, normalized relational model, synchronous. Query side: handles reads, eventual consistency, denormalized read models (optimised per query pattern), can use different DB type (ES for search, Redis for leaderboard). Sync strategies: (1) domain events → update read model asynchronously, (2) CDC (Debezium), (3) dual-write (risk). Trade-offs: read performance vs complexity, eventual consistency lag
15.6	Saga Pattern	Choreography Saga: each service publishes event after step completes, next service listens and acts. Pros: no central coordinator, decoupled. Cons: harder to trace, no single view of saga state, compensating tx coordination complex. Orchestration Saga: Saga Orchestrator sends commands to services, receives replies, maintains saga state (STARTED/STEP_1_DONE/FAILED). Pros: clear flow, easy to monitor. Cons: orchestrator is a coordination bottleneck. Compensating transactions: semantically undo completed steps

No.	Sub-Chapter Title	Key Topics Covered
15.7	Resilience Patterns	Circuit Breaker (Resilience4j): CLOSED (normal, count failures) → OPEN (fail fast, no calls, save resources) → HALF-OPEN (one probe call) → CLOSED if success. Retry: @Retry(maxAttempts=3, backoff=@Backoff(delay=1000,multiplier=2,jitter=500)). Bulkhead: ThreadPoolBulkhead (separate pool per downstream, failure isolation), SemaphoreBulkhead. Timeout: @TimeLimiter. Fallback: return cached/default. Rate Limiter. Chaos Engineering: Chaos Monkey for Spring Boot
15.8	API Versioning Strategies	URI versioning: /api/v1/users (most explicit, breaks REST resource concept). Query param: /users?version=1 (messy). Header: Accept: application/vnd.myapp.v1+json (cleanest, less discoverable). Backward compatibility: additive changes (new fields) are safe, never remove fields, use @JsonIgnoreUnknown. Deprecation: Deprecation header + Sunset header (RFC 8594). Consumer-driven contract testing (Pact): consumer defines expected API, provider verifies
15.9	Microservices Testing Strategies	Test pyramid: many unit → fewer integration → few E2E. Unit: mock all dependencies, fast, isolated. Integration: Testcontainers (real DB/Redis/Kafka), test single service. Component: deploy service + its dependencies (Docker Compose), test via HTTP. Contract (Pact): consumer writes contract (expected request/response), provider verifies against contract in CI. E2E: minimal, slow, fragile – only for critical happy paths. Consumer-driven contracts prevent provider breaking consumers
15.10	Microservices Anti-patterns & Migration	Distributed Monolith: microservices but tightly coupled (shared DB, sync chains, coordinated deployment). Chatty services (too many small calls, combine with GraphQL or BFF). Shared DB (breaks service independence, schema changes break all services). Migration: Strangler Fig (route traffic feature-by-feature to new service), parallel run (both systems active, compare results), feature flags for cutover. Data migration: dual-write → backfill → cutover → cleanup

Phase 6: Infrastructure & Deployment

Chapter 16 Docker – Containers & Images (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
16.1	Docker Architecture	Docker daemon (dockerd, REST API), Docker CLI (sends commands to daemon), containerd (container runtime, manages container lifecycle), runc (OCI runtime, creates containers via Linux namespaces+cgroups). OCI: image spec (layers + config + manifest) + runtime spec. Union filesystem (OverlayFS): layers stacked, upperdir = writable container layer. Container vs VM: container shares OS kernel, VM has full OS
16.2	Dockerfile & Image Best Practices	FROM (choose minimal: distroless, Alpine, Eclipse Temurin). RUN (combine with && to reduce layers). COPY (prefer over ADD for local files). CMD (default command, overridable) vs ENTRYPOINT (fixed executable). ENV/ARG (ARG build-time only). EXPOSE (documentation only, doesn't publish). USER nonroot (security). .dockerignore (exclude .git, target/, node_modules). Layer caching: COPY pom.xml + RUN mvn dependency:go-offline before COPY src/
16.3	Multi-Stage Builds	Stage 1 (builder): FROM maven:3.9-eclipse-temurin-21 AS builder, COPY pom.xml + src, RUN mvn package -DskipTests. Stage 2 (runtime): FROM eclipse-temurin:21-jre-jammy, COPY --from=builder /app/target/*.jar /app/app.jar, ENTRYPOINT ["java","-jar","/app/app.jar"]. Result: image contains only JRE + JAR (no Maven/sources/tests). Size reduction: 600MB → 200MB. --target flag to build specific stage
16.4	Docker Networking	Bridge (default, NAT, docker0 interface, container-to-container via IP). User-defined bridge: containers resolve each other by name (DNS). Host: no network isolation, container uses host network stack (Linux only, better performance). Overlay: multi-host networking (Docker Swarm, encrypted), containers on different hosts communicate. None: loopback only. Port mapping: -p 8080:8080 (host:container). Service names as DNS in docker-compose
16.5	Docker Volumes & Storage	Named volumes: docker volume create mydata, persistent across container restarts, managed by Docker. Bind mounts: map host path to container path (-v /host/path:/container/path), good for development (live code reload). tmpfs: in-memory, not persisted, for sensitive data. Volume drivers (nfs/local/cloud). Volume backup: docker run --volumes-from container -v \$(pwd):/backup ubuntu tar backup. Docker secrets (Swarm): stored in Raft log, mounted as files, encrypted at rest and in transit
16.6	Docker Compose	docker-compose.yml: services (image/build/ports/volumes/environment/depends_on/networks/healthcheck), volumes (named), networks (custom bridge). depends_on with condition: service_healthy (requires healthcheck). .env file for variables. Profiles (--profile prod): conditional services. Override files: docker-compose.yml + docker-compose.override.yml merged (dev overrides). docker compose watch (file sync + automatic restart for dev)
16.7	Docker Security	Non-root USER (addgroup --system app && adduser --system --ingroup app app). Read-only filesystem (--read-only + --tmpfs /tmp). Capability dropping (--cap-drop ALL --cap-add NET_BIND_SERVICE). Seccomp profiles (default blocks 44 syscalls). Image scanning: Trivy (all layers, CVE database), Snyk Container, Docker Scout. Signed images (Docker Content Trust / Notary, Cosign + Sigstore). Resource limits (--memory 512m --cpus 1)
16.8	Container Registry	Docker Hub (public, rate-limited pulls – 100 pulls/6h anonymous). AWS ECR: IAM-based auth (aws ecr get-login-password docker login), lifecycle policies (delete untagged after 7 days), image scanning on push (Clair/Snyk). GCR → Artifact Registry (multi-format: Docker/Maven/npm/Python, IAM, automatic vulnerability scanning). Harbor: self-hosted, RBAC, project quotas, image replication, Helm chart repo, OIDC SSO

No.	Sub-Chapter Title	Key Topics Covered
16.9	Docker Without Kubernetes	Docker Swarm: built-in orchestration (docker swarm init, docker service create, rolling updates with --update-delay, overlay networking). Swarm vs K8s: Swarm is simpler (same CLI, less features); K8s is more powerful (auto-scaling, richer ecosystem). AWS ECS + Fargate: serverless containers (no EC2 management, task definition, ECS service, ALB integration, CloudWatch logs, IAM task roles). Azure Container Instances (ACI): on-demand, per-second billing
16.10	Debugging & Troubleshooting Containers	docker logs -f --tail 100 . docker exec -it sh (enter running container). docker inspect (full JSON config: env/mounts/network/state). docker stats (CPU%/MEM/NET/BLOCK real-time). docker top (processes). Ephemeral debug container: docker run --rm -it --net container: nicolaka/netshoot (network tools). OOMKilled: check docker inspect .State.OOMKilled + increase --memory. Healthcheck debugging: docker inspect .State.Health.Log

Chapter 17 Kubernetes – Orchestration & Deployment (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
17.1	Kubernetes Architecture	Control Plane: kube-apiserver (REST API, auth/authz, admission controllers), etcd (distributed KV store, cluster state, Raft consensus), kube-scheduler (assigns pods to nodes, scoring: resource fit/affinity/taints), kube-controller-manager (Deployment/ReplicaSet/Node/EndpointSlice controllers). Node: kubelet (ensures containers running, liveness/readiness probes), kube-proxy (iptables/IPVS rules for Service VIP), container runtime (containerd via CRI). CNI (Calico/Cilium/Flannel). CSI (EBS/PD drivers)
17.2	Core Workloads	Pod: smallest unit, shared network namespace (localhost), shared volumes, init containers (run to completion before main). Deployment: manages ReplicaSet, rolling update (maxUnavailable/maxSurge), rollback (kubectl rollout undo), revision history (revisionHistoryLimit). StatefulSet: stable hostname (pod-0/pod-1), stable storage (volumeClaimTemplates), ordered startup/shutdown (needed for Kafka/Cassandra). DaemonSet: one pod per node (log collection, monitoring). Job/CronJob (schedule: '0 */6 * * *')
17.3	Services & Cluster DNS	ClusterIP (default): stable virtual IP, kube-proxy maintains iptables rules to pod IPs, DNS: ..svc.cluster.local. NodePort: exposes on each node's IP:port (30000-32767), ClusterIP included. LoadBalancer: cloud LB provisioned, external IP assigned (AWS NLB/ELB, GCP GCLB), NodePort included. ExternalName: CNAME to external DNS. Headless (clusterIP: None): DNS returns pod IPs directly (StatefulSet, Cassandra discovery). EndpointSlice (scalable alternative to Endpoints)
17.4	Ingress & TLS Termination	Ingress resource: routing rules (spec.rules: host + http.paths: path+pathType+backend.service.name/port). Nginx Ingress Controller: Deployment + LoadBalancer Service, reads Ingress objects via informer, generates nginx.conf. Annotations: nginx.ingress.kubernetes.io/rewrite-target, rate-limit, auth-url, proxy-body-size. cert-manager: ClusterIssuer (Let's Encrypt ACME), Certificate resource auto-provisioned from Ingress annotation (cert-manager.io/cluster-issuer). Gateway API (next-gen): HTTPRoute, GatewayClass, Gateway
17.5	ConfigMaps & Secrets	ConfigMap: non-sensitive config. As env vars (envFrom: configMapRef), specific key (env: valueFrom: configMapKeyRef), volume mount (files in /config/). Secret: base64 encoded (NOT encrypted), types: Opaque/kubernetes.io/tls/kubernetes.io/dockerconfigjson. External Secrets Operator: syncs from AWS Secrets Manager/GCP Secret Manager/Vault to K8s Secret (auto-refresh). Sealed Secrets (Bitnami): encrypted SealedSecret in Git, controller decrypts. Vault Agent Injector: sidecar injects secrets as files
17.6	Autoscaling	HPA: scales replicas based on metrics. CPU (metrics-server required): averageUtilization=70. Custom/external metrics via metrics-server + KEDA (Kafka consumer lag, SQS queue depth, Prometheus metric). VPA: recommends/sets resource requests (updateMode: Auto[Off]Initial). Cluster Autoscaler: provisions/removes nodes based on pending pods + idle nodes (cloud provider node groups). KEDA (Kubernetes Event-Driven Autoscaling): scale-to-zero, ScaledObject/ScaledJob CRDs, 60+ scalers

No.	Sub-Chapter Title	Key Topics Covered
17.7	Kubernetes Storage	PersistentVolume (PV): cluster resource representing storage (NFS/EBS/PD/local). PersistentVolumeClaim (PVC): pod's request for storage (storageClassName/accessMode/storage). Dynamic provisioning via StorageClass (provisioner: ebs.csi.aws.com, parameters: type=gp3, reclaimPolicy: Retain). Access modes: ReadWriteOnce (single node), ReadOnlyMany (multiple nodes read), ReadWriteMany (multiple nodes write – NFS/EFS/GCS). StatefulSet volumeClaimTemplates (each pod gets own PVC)
17.8	Helm & Package Management	Chart: Chart.yaml (name/version/appVersion), values.yaml (defaults), templates/ (Go template YAML). Helm install/upgrade/rollback/uninstall. Values override: --set image.tag=v2.0 or -f prod-values.yaml (multiple allowed). Helm hooks: pre-install (DB migration Job), post-upgrade (smoke test), pre-delete. Helmfile: declarative multi-chart management (releases/repos/environments). Helm test (helm test runs test pods). OCI registry for chart storage. Kustomize (alternative, built into kubectl)
17.9	GKE & EKS – Managed Kubernetes	GKE: Standard (you manage nodes) vs Autopilot (Google manages nodes, pay per pod, no node access). Node pools (different machine types per pool). Workload Identity (pod SA → GCP SA, no key files). Binary Authorization (only signed images). Cloud Armor (WAF + DDoS). EKS: managed control plane (HA etcd, version upgrades). Managed node groups (EC2 ASG). Fargate profiles (serverless pods). IRSA (IAM Roles for Service Accounts, OIDC token exchange). AWS Load Balancer Controller (Ingress → ALB, Service → NLB)
17.10	Kubernetes Java API – Fabric8 Client	Fabric8 KubernetesClient: auto-configured from in-cluster ServiceAccount or kubeconfig. List pods: client.pods().inNamespace('ns').list(). Watch deployments: client.apps().deployments().watch(new Watcher<>(){...}). Scale: client.apps().deployments().inNamespace('ns').withName('app').scale(5). Create/patch resources (PatchType.MERGE_PATCH / SERVER_SIDE_APPLY). Informers: SharedIndexInformer for cache+watch. Leader election (LeaderElector for HA controllers). Custom Resource: client.resources(MyResource.class)

Chapter 18 Nginx & Reverse Proxy (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
18.1	Nginx Architecture & Event Model	Master process (reads config, manages workers, zero-downtime reload). Worker processes (handle connections, event-driven, non-blocking, single thread handles 1000s of connections). epoll (Linux) / kqueue (BSD/macOS). worker_processes auto (= CPU count). worker_connections 1024 (max connections per worker). request processing phases: post-read/server-rewrite/ind-config/rewrite/post-rewrite/preaccess/access/post-access/precontent/content/log
18.2	Reverse Proxy Configuration	proxy_pass http://backend:8080; (forward request to backend). upstream backend { server app1:8080; server app2:8080; } (pool). proxy_set_header Host \$host; X-Real-IP \$remote_addr; X-Forwarded-For \$proxy_add_x_forwarded_for; X-Forwarded-Proto \$scheme. proxy_read_timeout 60s; proxy_connect_timeout 10s. proxy_buffering on (buffer response, release backend faster). proxy_http_version 1.1; proxy_set_header Connection "" (keepalive to upstream)
18.3	Load Balancing Algorithms	Round robin (default, sequential). Least connections (least_conn; send to server with fewest active connections, better for long requests). IP Hash (ip_hash; client IP always routes to same server – session affinity/sticky sessions). Hash \$request_uri (URL-based affinity). Random (random; with two = pick best of 2 random). Weights: server backend1 weight=3 (3× more traffic). Backup: server backup1 backup (only if all others down). Passive health check: fail_timeout=30s max_fails=3
18.4	SSL/TLS Termination	ssl_certificate /etc/nginx/certs/fullchain.pem; ssl_certificate_key /etc/nginx/certs/privkey.pem; listen 443 ssl http2; ssl_protocols TLSv1.2 TLSv1.3; ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:... (Mozilla modern config). ssl_session_cache shared:SSL:10m; ssl_session_timeout 1d. HSTS: add_header Strict-Transport-Security 'max-age=31536000; includeSubDomains; preload'. OCSP stapling: ssl_stapling on. Redirect HTTP→HTTPS: return 301 https://\$host\$request_uri

No.	Sub-Chapter Title	Key Topics Covered
18.5	Rate Limiting & Security	limit_req_zone \$binary_remote_addr zone=api:10m rate=10r/s; limit_req zone=api burst=20 nodelay; (allow burst of 20, no delay, then 429). limit_conn_zone \$binary_remote_addr zone=conn:10m; limit_conn conn 10; (max 10 simultaneous connections per IP). Geo module: geo \$whitelist { default 0; 10.0.0.0/8 1; } (allow internal IPs). return 444 (Nginx-specific, close connection without response). ModSecurity WAF (naxsi alternative, CRS rules for SQLi/XSS)
18.6	Location Matching & Routing	Priority: = (exact match, highest) > ^~ (prefix, stops regex) > ~ (case-sensitive regex) > ~* (case-insensitive regex) > / (prefix catch-all, lowest). Example: location = /health { return 200; } location ^~ /static/ { root /var/www; } location ~* \.php\$ { return 403; } location / { proxy_pass http://app; } Named locations: location @fallback { proxy_pass http://fallback; } try_files \$uri \$uri/ @fallback; Error routing: error_page 503 @maintenance
18.7	Nginx as Kubernetes Ingress	Nginx Ingress Controller: Deployment (ingress-nginx-controller) + LoadBalancer Service + ClusterRole. Reads Ingress objects via K8s informer, generates /etc/nginx/nginx.conf, does graceful reload. Annotations: nginx.ingress.kubernetes.io/rewrite-target: / (strip prefix), /proxy-body-size: 10m, /rate-limit: '10', /auth-url: 'http://auth-service/validate', /canary: 'true' + /canary-weight: '10' (10% to canary). Global config via ConfigMap (use-forwarded-headers, worker-processes)
18.8	Nginx Caching	proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api_cache:10m max_size=1g inactive=60m use_temp_path=off. proxy_cache api_cache; proxy_cache_key '\$scheme\$request_method\$host\$request_uri'; proxy_cache_valid 200 10m; proxy_cache_valid 404 1m. Cache bypass: proxy_cache_bypass \$http_pragma \$http_authorization. Add header: add_header X-Cache-Status \$upstream_cache_status (HIT/MISS/BYPASS). Purge: ngx_cache_purge module (PURGE method)
18.9	Nginx vs Alternatives	HAProxy: L4+L7 LB, pure proxy (no web server), excellent health checks (HTTP/TCP/SSL/custom scripts), ACL-based routing, active-active HA (keepalived+VRRP), best stats page. Traefik: cloud-native, auto-discovers routes via Docker labels/K8s annotations/Consul, Let's Encrypt built-in, dashboard, middleware (rate limit/auth/compress/retry), proxy protocol. Envoy: L7 proxy, xDS dynamic config (no reload), service mesh sidecar (Istio), gRPC-first, powerful observability (Prometheus metrics, access logs)
18.10	Monitoring & Debugging Nginx	stub_status: location /nginx_status { stub_status; } (active connections/accepted/handled/requests/reading/writing/waiting). Access log JSON format: log_format json escape=json '{"time": "\$time_iso8601", "status": \$status, "request_time": \$request_time,...}'. error_log /var/log/nginx/error.log warn. nginx -t (config syntax test). nginx -s reload (graceful reload, no connection drop). Tuning: worker_rlimit_nofile 65535; keepalive_requests 1000; keepalive_timeout 65. open_file_cache max=1000 inactive=20s

Chapter 19 DevOps – CI/CD, AWS & GCP (10 sub-chapters)

No.	Sub-Chapter Title	Key Topics Covered
19.1	CI/CD Pipeline Design	GitHub Actions: workflow (on push/PR/schedule), jobs (parallel or sequential with needs:), steps (run shell/uses action), matrix (Java 17+21, OS linux+mac), reusable workflows (workflow_call), environments (approval gates for prod), OIDC token (keyless cloud auth, no long-lived secrets). Jenkins: Declarative Pipeline (stages/steps/post/when), shared libraries, multi-branch pipelines, Blue Ocean UI. GitLab CI: stages/jobs, rules (if/changes), artifacts (pass between jobs), environments, cache
19.2	Infrastructure as Code	Terraform: providers (AWS/GCP/Azure), resources (aws_instance/google_container_cluster), data sources (existing resources), modules (reusable components), state (remote: S3+DynamoDB lock / GCS), workspaces (dev/staging/prod), plan (dry run) / apply / destroy. Taint/refresh/import. Ansible: playbooks (YAML tasks), inventory (static/dynamic), roles (reusable, Galaxy), idempotency, Vault integration for secrets. Pulumi: IaC in Python/TypeScript/Java (familiar language, real loops/conditions)

No.	Sub-Chapter Title	Key Topics Covered
19.3	AWS Core Services for Java Apps	EC2: instance families (t3/m5/c5/r5/i3 – burstable/general/compute/memory/storage optimised), launch templates, ASG (desired/min/max, scaling policies, lifecycle hooks). ECS: task definition (container image/CPU/memory/env/secrets/log config), ECS service (desired count, deployment type, LB integration), Fargate (no EC2 management, per-vCPU+GB/s billing). EKS: managed control plane, managed node groups (ASG-backed), add-ons (CoreDNS/kube-proxy/VPC CNI/EBS CSI)
19.4	AWS Storage & Messaging	S3: buckets (globally unique, region-specific), presigned URLs (time-limited, no auth header needed), lifecycle rules (transition to IA/Glacier, expire after N days), Transfer Acceleration (CloudFront edge upload), multipart upload (>100MB), S3 Events → Lambda/SQS. SQS: standard (at-least-once, best-effort order), FIFO (exactly-once, ordered, 300 TPS or 3000 with batching), DLQ (after maxReceiveCount), visibility timeout (hide during processing). SNS: pub/sub fan-out to SQS/Lambda/HTTP/Email/SMS, message filtering
19.5	AWS Advanced – Serverless & Edge	Lambda: runtime (Java 21 with SnapStart – restore from snapshot, fast cold start), handler (RequestHandler), layers (shared dependencies), concurrency (reserved/provisioned), cold start mitigation (SnapStart, provisioned concurrency, keep-warm). API Gateway: REST API (stages/usage plans/throttling) vs HTTP API (faster, cheaper, JWT authorizer). CloudFront: distributions, origin access control, cache behaviors, WAF integration, real-time logs
19.6	AWS Data Services	DynamoDB: on-demand (pay per request) vs provisioned (RCU/WCU, auto-scaling). DynamoDB Streams + Lambda (react to changes, fan-out). Global Tables (multi-region active-active). ElastiCache: Redis cluster mode enabled (sharding + HA) vs disabled (single shard + replicas). Global Datastore (cross-region replica). Aurora: compatible with MySQL/PostgreSQL, 6-way replication across 3 AZs, Aurora Serverless v2 (instant scaling to 0), global database (RPO=1s, RTO=<1min)
19.7	GCP Core Services for Java	GKE: Standard (manage node pools) vs Autopilot (fully managed nodes, pod-level billing, hardened security). Workload Identity Federation (pod ServiceAccount → GCP IAM, no key files). Cloud Run: fully managed containers, HTTP trigger, min-instances (avoid cold start), concurrency per container (up to 1000), --vpc-connector (access VPC resources), Cloud Run Jobs (batch). Cloud SQL: PostgreSQL/MySQL, HA (primary + standby in different zone), read replicas, Cloud SQL Auth Proxy (IAM auth, no IP allowlisting)
19.8	GCP Messaging & Data	Pub/Sub: topics + subscriptions (push to HTTP endpoint or pull by consumer), at-least-once delivery, message ordering (ordering key, single partition), dead-letter topics (after maxDeliveryAttempts), snapshot/seek (replay messages), Pub/Sub Lite (zonal, cheaper, lower durability). Firestore: document-collection model, real-time listeners (onSnapshot), offline support, ACID transactions, composite indexes, Cloud Firestore vs Realtime Database. BigQuery: serverless OLAP, columnar, petabyte scale, partitioning (date/integer), clustering, federated queries (GCS/Bigtable/Sheets), BigQuery ML
19.9	Observability & SRE Practices	Prometheus: scrape (HTTP /metrics, Micrometer auto-exposes JVM+Spring metrics), PromQL (rate(http_requests_total[5m]), histogram_quantile(0.95, ...)), recording rules (pre-compute expensive queries), Alertmanager (routing/grouping/inhibition/silence, PagerDuty/Slack receivers). Grafana: dashboards, alerts, annotations. OpenTelemetry: Java auto-instrumentation agent (java -javaagent:otel-agent.jar), OTLP export to Jaeger/Zipkin/Tempo. SLI (latency p99 < 200ms, error rate < 0.1%), SLO (monthly), SLA (contractual), error budget
19.10	Deployment Strategies & DevSecOps	Blue-Green: two identical envs, switch DNS/LB (instant cutover, full double resources, easy rollback). Canary: route X% traffic to new version (5%→25%→100%), metric-based promotion (error rate, latency), Flagger (K8s operator automates canary). Rolling: gradual pod replacement (maxUnavailable=1, maxSurge=1), zero downtime. Feature flags: LaunchDarkly/Unleash (decouple deploy from release, A/B test, kill switch). DevSecOps: SAST (SpotBugs, SonarQube in CI), DAST (OWASP ZAP), SCA (Dependabot/Snyk), container signing (Cosign, Sigstore)

Complete Sub-Chapter Index (193 entries)

No.	Sub-Chapter Title	Phase
1.1	Core Java & JVM Internals — Java Primitives, Wrappers & Strings	Phase 1
1.2	Core Java & JVM Internals — Collections – Framework Overview & List	Phase 1
1.3	Core Java & JVM Internals — Collections – Set Implementations	Phase 1
1.4	Core Java & JVM Internals — Collections – Map Implementations	Phase 1
1.5	Core Java & JVM Internals — Collections – Queue, Deque & BlockingQueue	Phase 1
1.6	Core Java & JVM Internals — Collections – Internals, Algorithms & Best Practices	Phase 1
1.7	Core Java & JVM Internals — Generics & Type System	Phase 1
1.8	Core Java & JVM Internals — Exception Handling	Phase 1
1.9	Core Java & JVM Internals — Java 8 – Lambdas, Streams & Optional	Phase 1
1.10	Core Java & JVM Internals — Java 9–21 Modern Features	Phase 1
1.11	Core Java & JVM Internals — Multithreading Fundamentals	Phase 1
1.12	Core Java & JVM Internals — java.util.concurrent Utilities	Phase 1
1.13	Core Java & JVM Internals — JVM Internals – Class Loading, JIT & GC	Phase 1
2.1	Design Patterns & Design Principles — SOLID Principles	Phase 1
2.2	Design Patterns & Design Principles — Creational Patterns	Phase 1
2.3	Design Patterns & Design Principles — Structural Patterns – Part 1	Phase 1
2.4	Design Patterns & Design Principles — Structural Patterns – Part 2	Phase 1
2.5	Design Patterns & Design Principles — Behavioral Patterns – Part 1	Phase 1
2.6	Design Patterns & Design Principles — Behavioral Patterns – Part 2	Phase 1
2.7	Design Patterns & Design Principles — Behavioral Patterns – Part 3	Phase 1
2.8	Design Patterns & Design Principles — Enterprise & Architectural Patterns	Phase 1
2.9	Design Patterns & Design Principles — Domain-Driven Design (DDD) Basics	Phase 1
2.10	Design Patterns & Design Principles — Clean Architecture & Refactoring	Phase 1
3.1	Spring Framework Internals — IoC Container & Dependency Injection	Phase 2
3.2	Spring Framework Internals — Bean Lifecycle & Scopes	Phase 2
3.3	Spring Framework Internals — Spring AOP & Proxy Mechanism	Phase 2
3.4	Spring Framework Internals — Spring MVC & REST API	Phase 2
3.5	Spring Framework Internals — Spring Boot Auto-configuration & Actuator	Phase 2
3.6	Spring Framework Internals — Spring Data & Repository Pattern	Phase 2
3.7	Spring Framework Internals — Spring Transaction Management	Phase 2
3.8	Spring Framework Internals — Spring Security Architecture	Phase 2
3.9	Spring Framework Internals — Spring Events & Integration	Phase 2
3.10	Spring Framework Internals — Spring Testing	Phase 2
4.1	JPA & ORM Internals — JPA Entity Lifecycle & Persistence Context	Phase 2
4.2	JPA & ORM Internals — Hibernate Architecture	Phase 2
4.3	JPA & ORM Internals — Entity Mappings	Phase 2
4.4	JPA & ORM Internals — JPQL, Native Queries & Criteria API	Phase 2
4.5	JPA & ORM Internals — Lazy Loading & N+1 Problem	Phase 2
4.6	JPA & ORM Internals — JPA Caching – L1 & L2	Phase 2
4.7	JPA & ORM Internals — Optimistic & Pessimistic Locking	Phase 2
4.8	JPA & ORM Internals — Dirty Checking & Flush Modes	Phase 2
4.9	JPA & ORM Internals — Database Migrations – Flyway & Liquibase	Phase 2
4.10	JPA & ORM Internals — JPA Performance Tuning	Phase 2
5.1	SQL Databases & Query Optimization — SQL Fundamentals & DDL/DML	Phase 2
5.2	SQL Databases & Query Optimization — Indexes – Types & Internals	Phase 2
5.3	SQL Databases & Query Optimization — Query Optimization & EXPLAIN	Phase 2
5.4	SQL Databases & Query Optimization — Transactions & ACID	Phase 2

No.	Sub-Chapter Title	Phase
5.5	SQL Databases & Query Optimization — Isolation Levels & MVCC	Phase 2
5.6	SQL Databases & Query Optimization — Advanced SQL – Joins & Window Functions	Phase 2
5.7	SQL Databases & Query Optimization — Stored Procedures, Functions & Triggers	Phase 2
5.8	SQL Databases & Query Optimization — Table Partitioning	Phase 2
5.9	SQL Databases & Query Optimization — Replication & High Availability	Phase 2
5.10	SQL Databases & Query Optimization — PostgreSQL vs MySQL – Key Differences	Phase 2
6.1	NoSQL Databases — NoSQL Types & CAP Theorem	Phase 2
6.2	NoSQL Databases — Redis – Data Structures & Patterns	Phase 2
6.3	NoSQL Databases — Cassandra – Wide-Column Store	Phase 2
6.4	NoSQL Databases — DynamoDB – AWS Managed NoSQL	Phase 2
6.5	NoSQL Databases — MongoDB – Document Store	Phase 2
6.6	NoSQL Databases — Graph Databases – Neo4j	Phase 2
6.7	NoSQL Databases — Time-Series Databases	Phase 2
6.8	NoSQL Databases — NoSQL Data Modeling Patterns	Phase 2
6.9	NoSQL Databases — Choosing SQL vs NoSQL	Phase 2
6.10	NoSQL Databases — Cloud-Native & Multi-Model Databases	Phase 2
7.1	Build Tools – Maven & Gradle — Maven Build Lifecycle & POM	Phase 2
7.2	Build Tools – Maven & Gradle — Maven Dependency Management	Phase 2
7.3	Build Tools – Maven & Gradle — Maven Plugins	Phase 2
7.4	Build Tools – Maven & Gradle — Maven Multi-Module Projects	Phase 2
7.5	Build Tools – Maven & Gradle — Gradle Build Script & Tasks	Phase 2
7.6	Build Tools – Maven & Gradle — Gradle Dependency Management	Phase 2
7.7	Build Tools – Maven & Gradle — Gradle Multi-Module & Build Cache	Phase 2
7.8	Build Tools – Maven & Gradle — Gradle Plugins	Phase 2
7.9	Build Tools – Maven & Gradle — Maven vs Gradle – Deep Comparison	Phase 2
7.10	Build Tools – Maven & Gradle — Build Tool CI/CD Integration	Phase 2
8.1	Application Security — TLS/HTTPS & X.509 Certificates	Phase 3
8.2	Application Security — Authentication – Sessions, Cookies & JWT	Phase 3
8.3	Application Security — OAuth 2.0 Flows	Phase 3
8.4	Application Security — OpenID Connect (OIDC) & SSO	Phase 3
8.5	Application Security — SAML 2.0 & Enterprise SSO	Phase 3
8.6	Application Security — Spring Security Architecture	Phase 3
8.7	Application Security — API Security	Phase 3
8.8	Application Security — OWASP Top 10 & Injection Prevention	Phase 3
8.9	Application Security — Password Hashing & Cryptography	Phase 3
8.10	Application Security — RBAC, ABAC & Zero Trust	Phase 3
9.1	Caching – All Levels — CPU Caches & Memory Hierarchy	Phase 4
9.2	Caching – All Levels — JVM Object & Heap Caching	Phase 4
9.3	Caching – All Levels — Application-Level Caching	Phase 4
9.4	Caching – All Levels — Distributed Cache – Redis	Phase 4
9.5	Caching – All Levels — Distributed Cache – Hazelcast & Memcached	Phase 4
9.6	Caching – All Levels — Database Query Cache & Connection Pooling	Phase 4
9.7	Caching – All Levels — HTTP Caching	Phase 4
9.8	Caching – All Levels — CDN & Edge Caching	Phase 4
9.9	Caching – All Levels — Cache Strategies	Phase 4
9.10	Caching – All Levels — Cache Eviction Policies & Consistency	Phase 4
10.1	Search – Elasticsearch & Alternatives — Search Fundamentals & Inverted Index	Phase 4
10.2	Search – Elasticsearch & Alternatives — Elasticsearch Architecture	Phase 4
10.3	Search – Elasticsearch & Alternatives — Index Mappings & Analyzers	Phase 4

No.	Sub-Chapter Title	Phase
10.4	Search – Elasticsearch & Alternatives — Elasticsearch Query DSL	Phase 4
10.5	Search – Elasticsearch & Alternatives — Relevance Scoring & Boosting	Phase 4
10.6	Search – Elasticsearch & Alternatives — Autocomplete & Fuzzy Search	Phase 4
10.7	Search – Elasticsearch & Alternatives — Spring Data Elasticsearch	Phase 4
10.8	Search – Elasticsearch & Alternatives — Search Data Sync – CDC & Debezium	Phase 4
10.9	Search – Elasticsearch & Alternatives — Elasticsearch Performance Tuning	Phase 4
10.10	Search – Elasticsearch & Alternatives — Search Alternatives	Phase 4
11.1	Concurrency & Locking (Application Level) — Optimistic Concurrency Control (OCC)	Phase 4
11.2	Concurrency & Locking (Application Level) — Pessimistic Locking	Phase 4
11.3	Concurrency & Locking (Application Level) — ETags & HTTP Conditional Requests	Phase 4
11.4	Concurrency & Locking (Application Level) — Distributed Locks	Phase 4
11.5	Concurrency & Locking (Application Level) — Two-Phase Locking & Deadlocks	Phase 4
11.6	Concurrency & Locking (Application Level) — MVCC & Snapshot Isolation	Phase 4
11.7	Concurrency & Locking (Application Level) — Application-Level Conflict Resolution	Phase 4
11.8	Concurrency & Locking (Application Level) — Event Sourcing & Idempotency	Phase 4
11.9	Concurrency & Locking (Application Level) — CRDTs & Collaborative Editing	Phase 4
11.10	Concurrency & Locking (Application Level) — Database Lock Monitoring	Phase 4
12.1	i18n – Multi-Language, Currency & Timezone — Internationalization Fundamentals	Phase 4
12.2	i18n – Multi-Language, Currency & Timezone — Message Bundles & Resource Files	Phase 4
12.3	i18n – Multi-Language, Currency & Timezone — Number & Currency Formatting	Phase 4
12.4	i18n – Multi-Language, Currency & Timezone — Date & Time Localization	Phase 4
12.5	i18n – Multi-Language, Currency & Timezone — Timezone Handling in APIs & Databases	Phase 4
12.6	i18n – Multi-Language, Currency & Timezone — Character Encoding	Phase 4
12.7	i18n – Multi-Language, Currency & Timezone — RTL Language Support	Phase 4
12.8	i18n – Multi-Language, Currency & Timezone — Multi-Tenancy & Locale Per User	Phase 4
12.9	i18n – Multi-Language, Currency & Timezone — Translation Workflow & Management	Phase 4
12.10	i18n – Multi-Language, Currency & Timezone — Testing i18n Applications	Phase 4
13.1	Software Performance – Sync, Async & Reactive — Synchronous vs Asynchronous vs Semi-Sync	Phase 4
13.2	Software Performance – Sync, Async & Reactive — Reactive Programming – Project Reactor	Phase 4
13.3	Software Performance – Sync, Async & Reactive — Spring WebFlux & Non-Blocking IO	Phase 4
13.4	Software Performance – Sync, Async & Reactive — Backpressure & Flow Control	Phase 4
13.5	Software Performance – Sync, Async & Reactive — Thread Pool & Executor Tuning	Phase 4
13.6	Software Performance – Sync, Async & Reactive — Profiling & Performance Analysis	Phase 4
13.7	Software Performance – Sync, Async & Reactive — Load Testing	Phase 4
13.8	Software Performance – Sync, Async & Reactive — Memory Profiling & GC Tuning	Phase 4
13.9	Software Performance – Sync, Async & Reactive — JMH Benchmarking	Phase 4
13.10	Software Performance – Sync, Async & Reactive — Database Performance Tuning	Phase 4
14.1	System Design — System Design Methodology	Phase 5
14.2	System Design — Scalability Patterns	Phase 5
14.3	System Design — Consistent Hashing & Load Balancing	Phase 5
14.4	System Design — CAP Theorem & Consistency Models	Phase 5
14.5	System Design — Message Queues & Event Streaming	Phase 5
14.6	System Design — API Gateway & Service Discovery	Phase 5
14.7	System Design — Database at Scale – Sharding & CQRS	Phase 5
14.8	System Design — Observability – Metrics, Tracing & Logging	Phase 5
14.9	System Design — System Design Walkthroughs – Part 1	Phase 5
14.10	System Design — System Design Walkthroughs – Part 2	Phase 5
15.1	Microservices & Event-Driven Architecture — Monolith vs Microservices vs Modular Monolith	Phase 5
15.2	Microservices & Event-Driven Architecture — Service Decomposition & Bounded Contexts	Phase 5

No.	Sub-Chapter Title	Phase
15.3	Microservices & Event-Driven Architecture — REST vs gRPC vs GraphQL	Phase 5
15.4	Microservices & Event-Driven Architecture — Event-Driven Architecture	Phase 5
15.5	Microservices & Event-Driven Architecture — CQRS Pattern	Phase 5
15.6	Microservices & Event-Driven Architecture — Saga Pattern	Phase 5
15.7	Microservices & Event-Driven Architecture — Resilience Patterns	Phase 5
15.8	Microservices & Event-Driven Architecture — API Versioning Strategies	Phase 5
15.9	Microservices & Event-Driven Architecture — Microservices Testing Strategies	Phase 5
15.10	Microservices & Event-Driven Architecture — Microservices Anti-patterns & Migration	Phase 5
16.1	Docker – Containers & Images — Docker Architecture	Phase 6
16.2	Docker – Containers & Images — Dockerfile & Image Best Practices	Phase 6
16.3	Docker – Containers & Images — Multi-Stage Builds	Phase 6
16.4	Docker – Containers & Images — Docker Networking	Phase 6
16.5	Docker – Containers & Images — Docker Volumes & Storage	Phase 6
16.6	Docker – Containers & Images — Docker Compose	Phase 6
16.7	Docker – Containers & Images — Docker Security	Phase 6
16.8	Docker – Containers & Images — Container Registry	Phase 6
16.9	Docker – Containers & Images — Docker Without Kubernetes	Phase 6
16.10	Docker – Containers & Images — Debugging & Troubleshooting Containers	Phase 6
17.1	Kubernetes – Orchestration & Deployment — Kubernetes Architecture	Phase 6
17.2	Kubernetes – Orchestration & Deployment — Core Workloads	Phase 6
17.3	Kubernetes – Orchestration & Deployment — Services & Cluster DNS	Phase 6
17.4	Kubernetes – Orchestration & Deployment — Ingress & TLS Termination	Phase 6
17.5	Kubernetes – Orchestration & Deployment — ConfigMaps & Secrets	Phase 6
17.6	Kubernetes – Orchestration & Deployment — Autoscaling	Phase 6
17.7	Kubernetes – Orchestration & Deployment — Kubernetes Storage	Phase 6
17.8	Kubernetes – Orchestration & Deployment — Helm & Package Management	Phase 6
17.9	Kubernetes – Orchestration & Deployment — GKE & EKS – Managed Kubernetes	Phase 6
17.10	Kubernetes – Orchestration & Deployment — Kubernetes Java API – Fabric8 Client	Phase 6
18.1	Nginx & Reverse Proxy — Nginx Architecture & Event Model	Phase 6
18.2	Nginx & Reverse Proxy — Reverse Proxy Configuration	Phase 6
18.3	Nginx & Reverse Proxy — Load Balancing Algorithms	Phase 6
18.4	Nginx & Reverse Proxy — SSL/TLS Termination	Phase 6
18.5	Nginx & Reverse Proxy — Rate Limiting & Security	Phase 6
18.6	Nginx & Reverse Proxy — Location Matching & Routing	Phase 6
18.7	Nginx & Reverse Proxy — Nginx as Kubernetes Ingress	Phase 6
18.8	Nginx & Reverse Proxy — Nginx Caching	Phase 6
18.9	Nginx & Reverse Proxy — Nginx vs Alternatives	Phase 6
18.10	Nginx & Reverse Proxy — Monitoring & Debugging Nginx	Phase 6
19.1	DevOps – CI/CD, AWS & GCP — CI/CD Pipeline Design	Phase 6
19.2	DevOps – CI/CD, AWS & GCP — Infrastructure as Code	Phase 6
19.3	DevOps – CI/CD, AWS & GCP — AWS Core Services for Java Apps	Phase 6
19.4	DevOps – CI/CD, AWS & GCP — AWS Storage & Messaging	Phase 6
19.5	DevOps – CI/CD, AWS & GCP — AWS Advanced – Serverless & Edge	Phase 6
19.6	DevOps – CI/CD, AWS & GCP — AWS Data Services	Phase 6
19.7	DevOps – CI/CD, AWS & GCP — GCP Core Services for Java	Phase 6
19.8	DevOps – CI/CD, AWS & GCP — GCP Messaging & Data	Phase 6
19.9	DevOps – CI/CD, AWS & GCP — Observability & SRE Practices	Phase 6
19.10	DevOps – CI/CD, AWS & GCP — Deployment Strategies & DevSecOps	Phase 6

Java Backend Architecture Interview Series — 193 Sub-Chapters